Network Flow

Peerapon S.

CPE 231 Algorithms

Based on slides distributed with *Algorithm Design* by J. Kleinberg and É. Tardos. Addison-Wesley, 2005
http://www.cs.princeton.edu/~wayne/kleinberg-tardos/

Topics

Flow network and max-flow problem

Ford-Fulkerson method

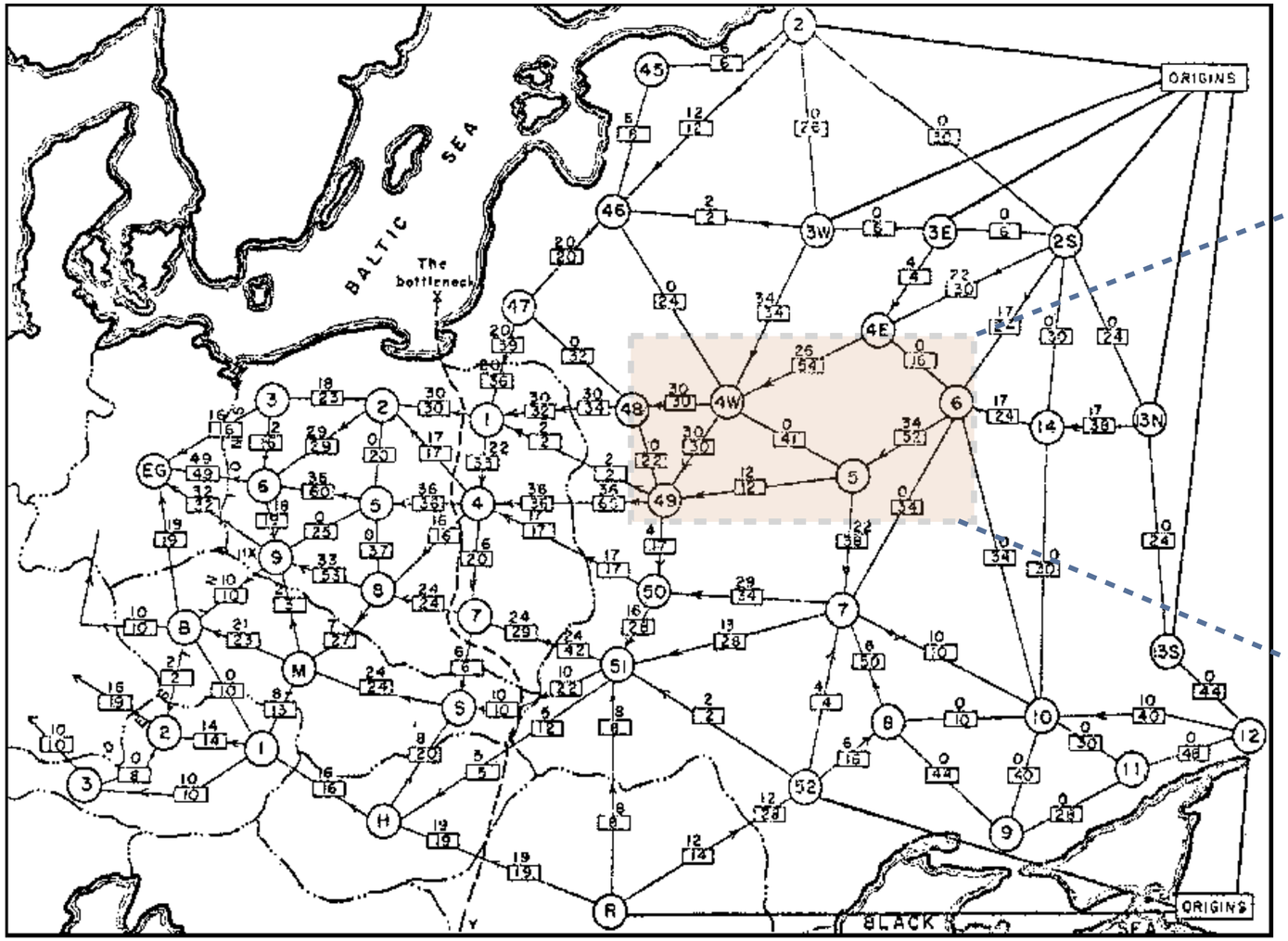
Cut and cut capacities

Max-flow min-cut theorem

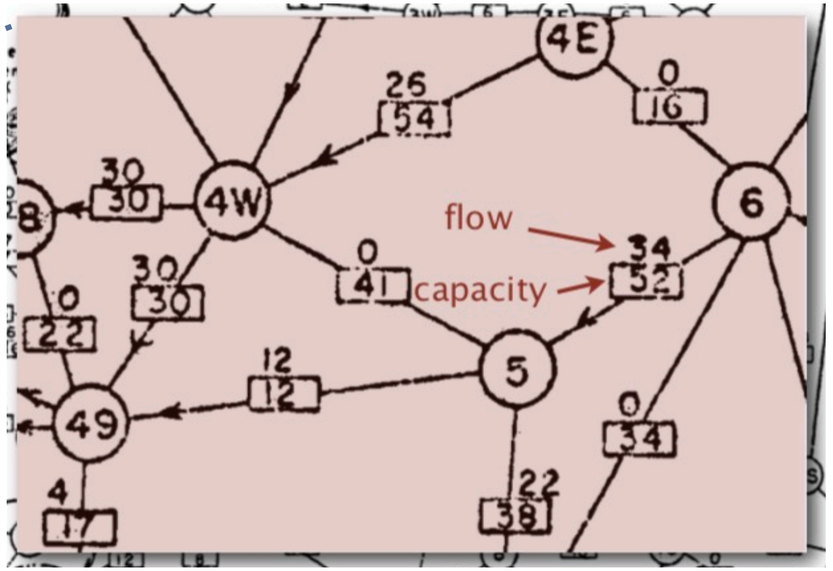
Network flow applications

Soviet Union goal. Maximize flow of supplies to Eastern Europe

"Free world" goal. Cheapest way to disrupt the largest amount of supplies



Harris and Ross's map of the Warsaw Pact rail network

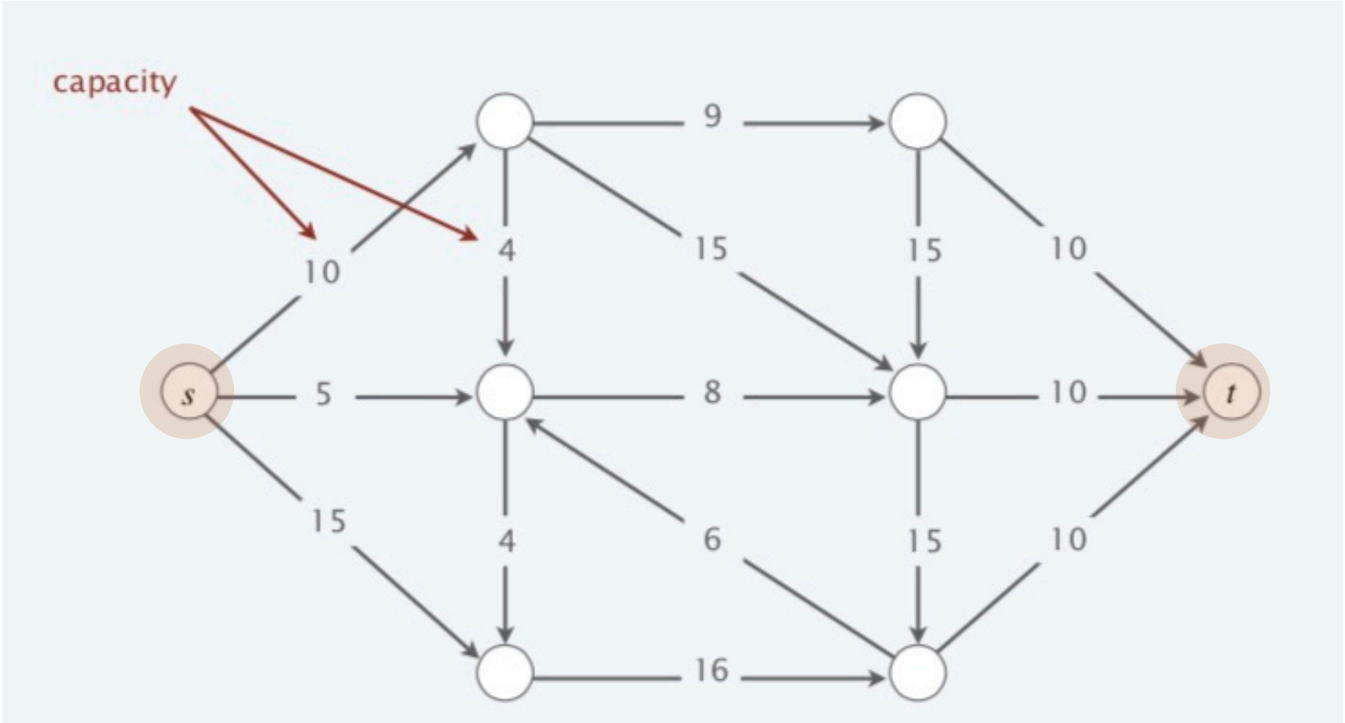


Flow Network

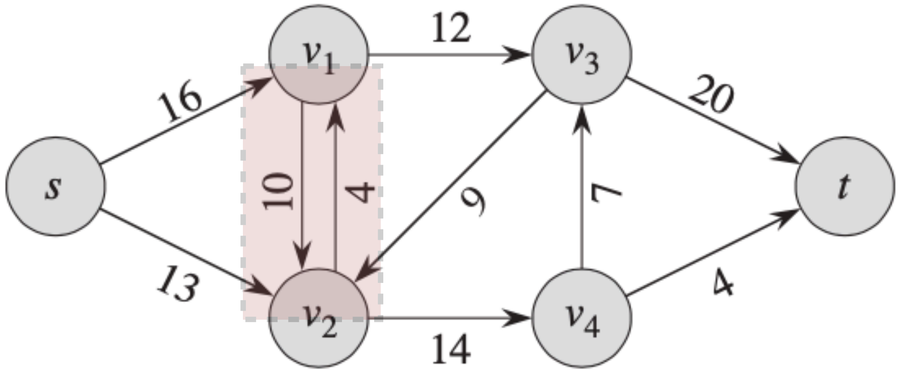
Flow network is a 5-tuple $\boldsymbol{G} = (\boldsymbol{V}, \boldsymbol{E}, s, t, c)$

- Digraph with no anti-parallel edge
- Source s and Terminal (sink) t nodes
- Positive-integer edge capacities

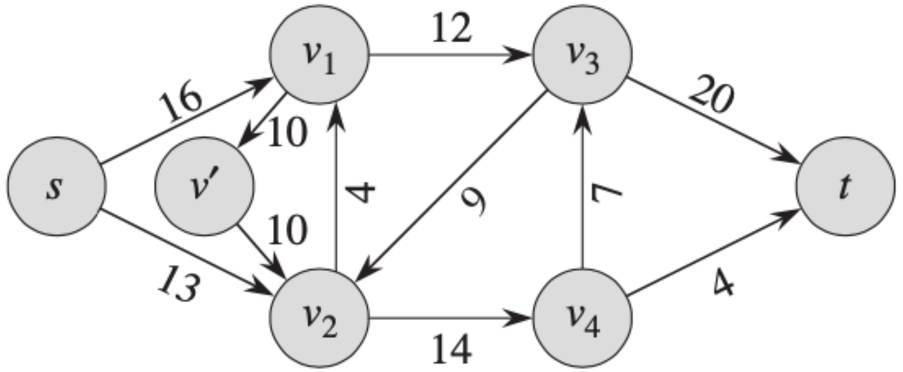
Also referred to as an st-network.



Dealing with Anti-Parallel Edges



(a)



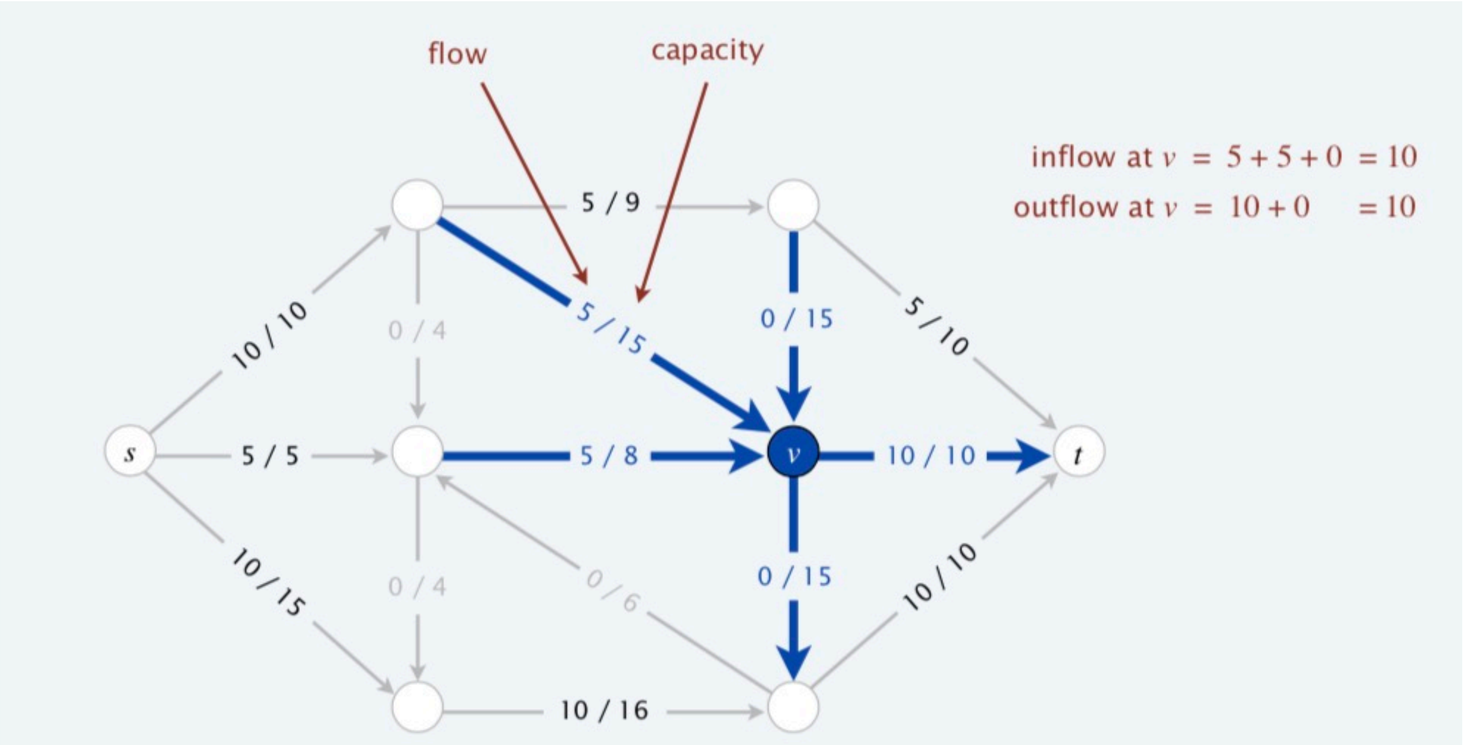
(b)

Edge Value Assignment $f(\cdot)$

Capacity constraint: Assignment of $f(e)$ such that $0 \leq f(e) \leq c(e)$ for $\forall e \in \boldsymbol{E}$

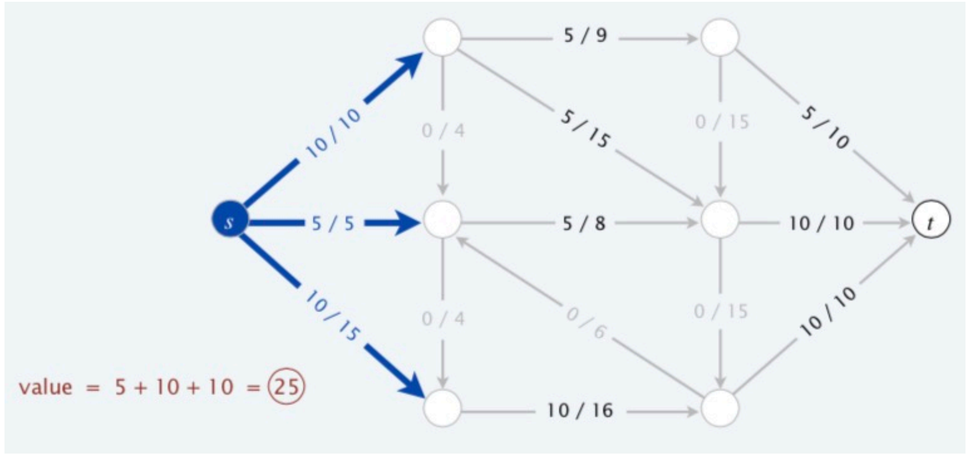
Flow conservation: Requirement at node v *(neither s nor t):*

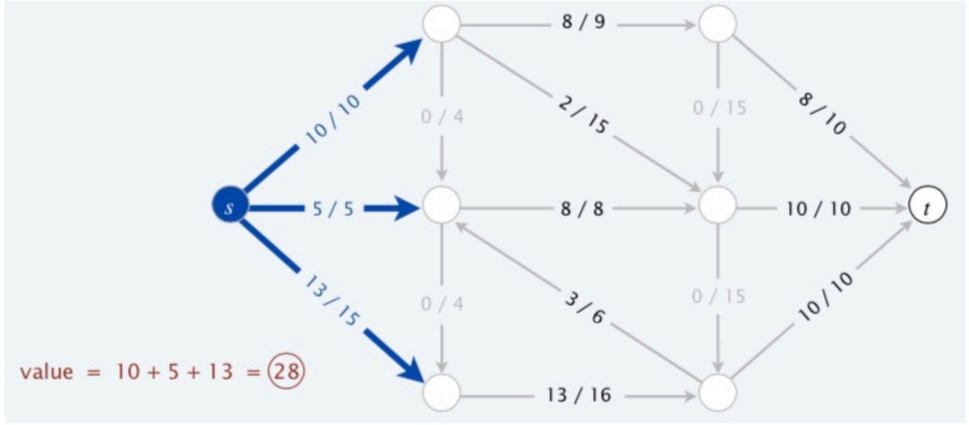
$$\sum_{e \text{ into } v} f(e) = \sum_{e \text{ out of } v} f(e)$$



Flow Value

$$v(f) = \sum_{e \text{ out of } s} f(e) = \sum_{e \text{ in to } t} f(e)$$





Maximum Flow (Max-Flow) Problem

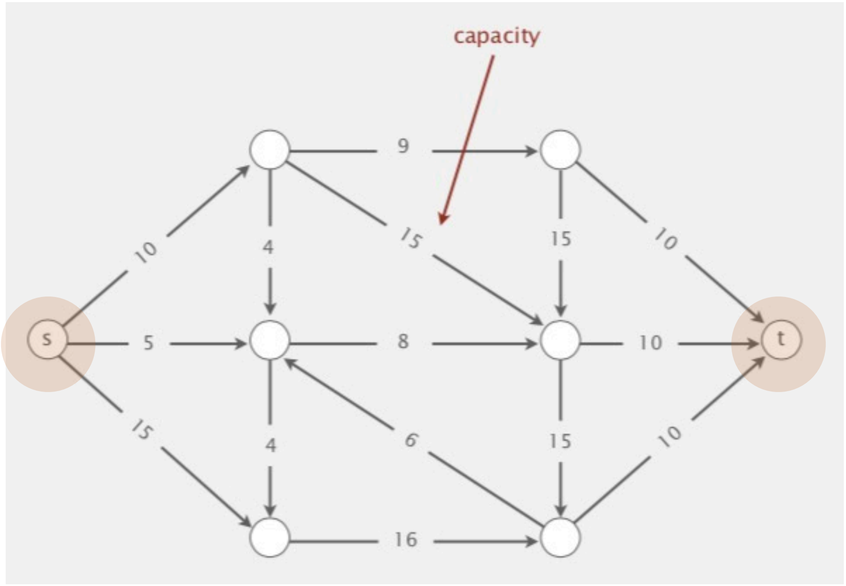
An *st*-flow (flow) is an *assignment* of values $f(e)$ to all the edges e such that:

- ***Capacity constraint***: $0 \le f(e) \le c(e)$
- ***Local equilibrium***: ***inflow*** $=$ ***outflow*** at every vertex (except s and t).

Def. The ***value*** of a flow $v(f)$ is the ***inflow*** at t, or equivalently the ***outflow*** at s.

Maximum st-flow (max-flow) problem. Find a flow of maximum value.

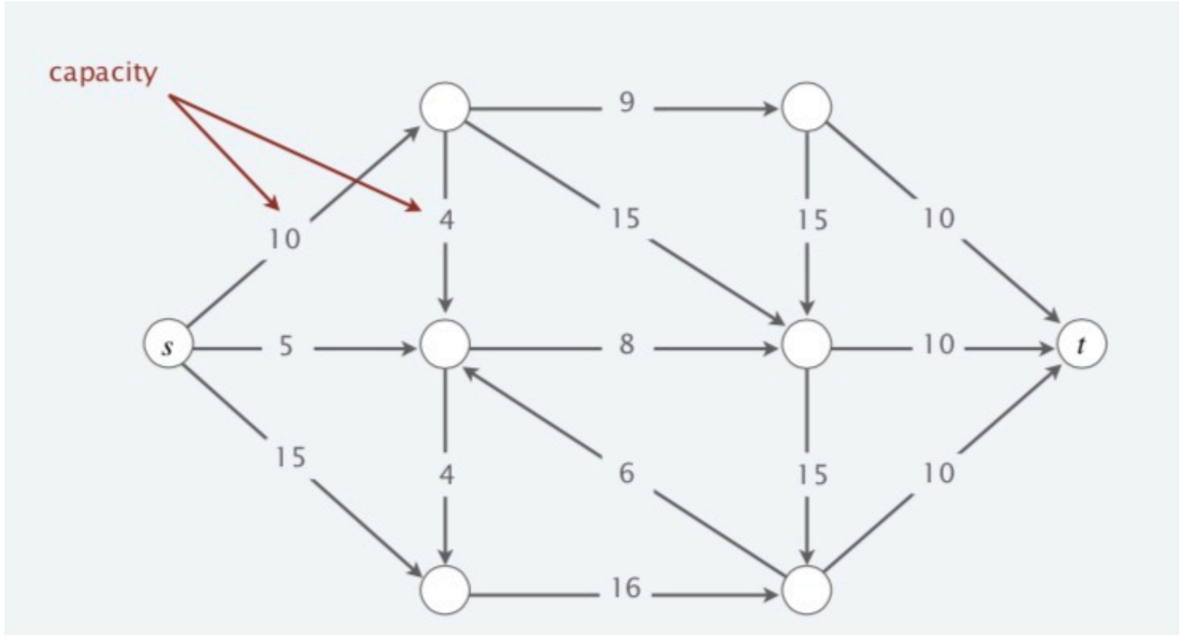
Consider a rail network connecting two cities by way of a number of intermediate cities, where each link of the network has a number assigned to it representing its capacity. Assuming a steady state condition, find a maximal flow from one given city to the other.



L.R. Ford, D.R. Fulkerson, Maximal Flow through a Network, Research Memorandum RM- 1400, The RAND Corporation, Santa Monica, California, [19 November] 1954
[published in Canadian Journal of Mathematics 8 (1956) 399–404].

Greedy Algorithm for Max-Flow Problem

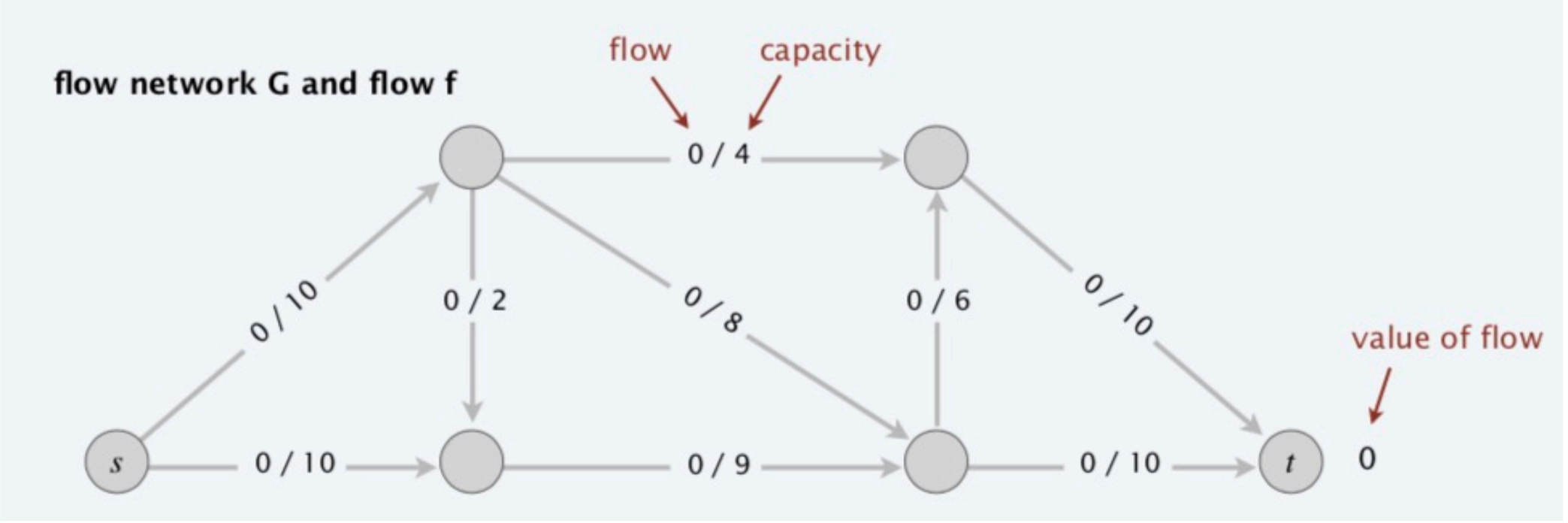
1. Start with $f(e) = 0$ for $\forall e \in \boldsymbol{E}$.
2. Find a simple $s \rightsquigarrow t$ path P where each edge has $f(e) < c(e)$.
3. Augment flow along path P.
4. Repeat step 2 until no path in step 2 can be found.



Self Test: Greedy Solution of Max-Flow Problem



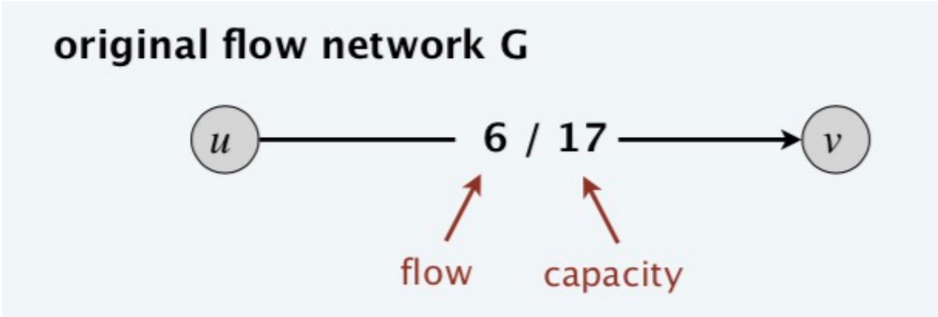
https://forms.office.com/r/DwjwVgynev?origin=lprLink


flow network G and flow f
flow
capacity
0 / 4
0 / 10
0 / 2
0 / 8
0 / 6
0 / 10
value of flow
s
0 / 10
0 / 9
0 / 10
t
0

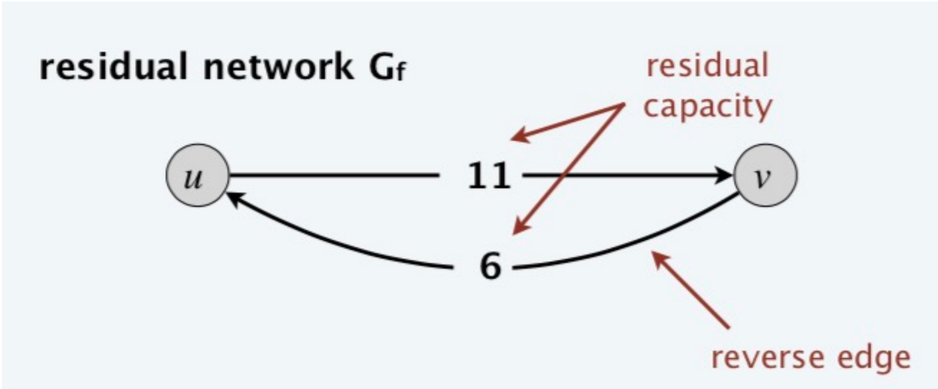
Residual Network

Forward edge capacity = Largest flow to push forward

Reverse edge capacity = Largest flow to push backward



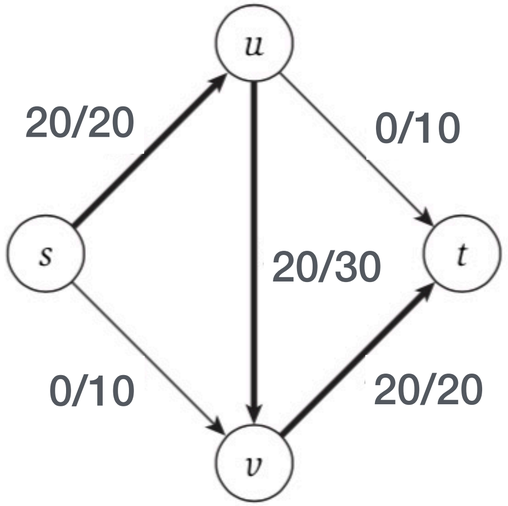
Original edge e
Flow $f(e) = 6$
Capacity $c(e) = 17$



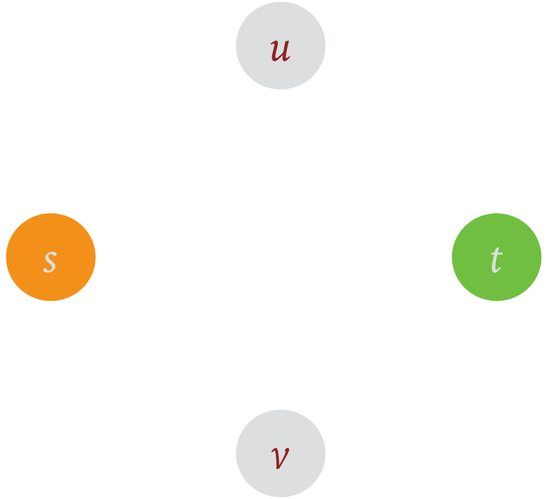
Forward edge $e = (u, v)$
with capacity $c(e) - f(e) > 0$

Backward edge $e' = (v, u)$
with capacity $f(e) > 0$

Graph G with flow f



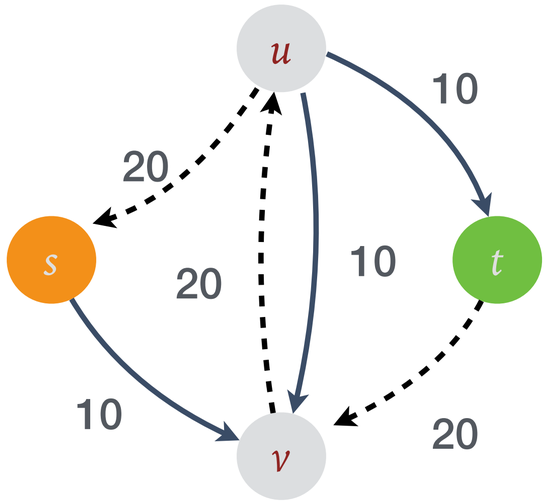
Residual graph G_f



Augmenting Path on Residual Graph

Let P be a *simple st* path on the residual graph.

Let $\delta = \text{bottleneck}(P, f)$ be the minimum capacity of any edge of P in $\boldsymbol{G}_f$



$P = s - v - u - t$

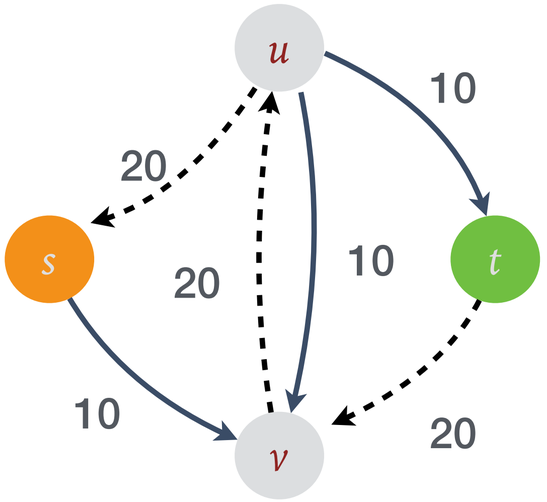
bottleneck (P, f) = ?

Push flow forward or backward by δ over edges on P

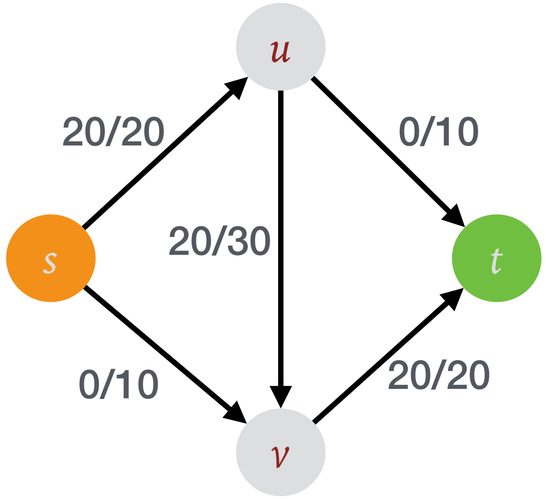
Operation Augment(f, P) yields a new flow f' in $\boldsymbol{G}$

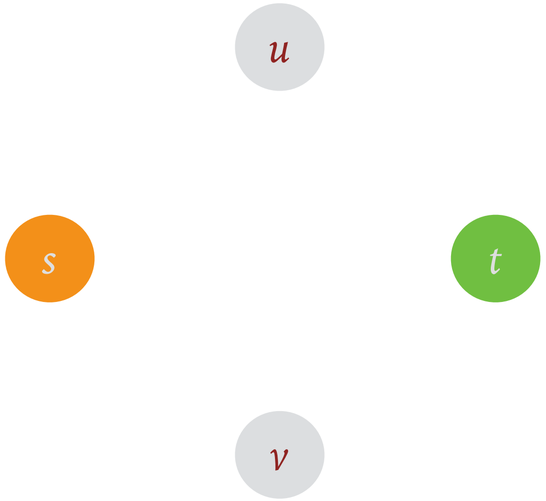
Algorithm Augment(f, P)

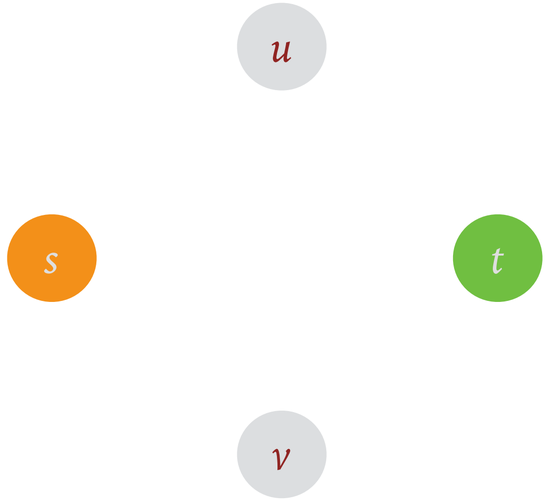
```
1:  Input: flow f on G and simple path P on G_f
2:  Output: New flow f' on G
3:
4:  δ ← bottleneck(P, f)
5:  for each edge e ∈ P do
6:      if e is a forward edge in G_f then
7:          Increase f(e) in G by δ
8:      else
9:          Let e' be the reverse of e
10:         Decrease f(e') in G by δ
11:
12: return f
```

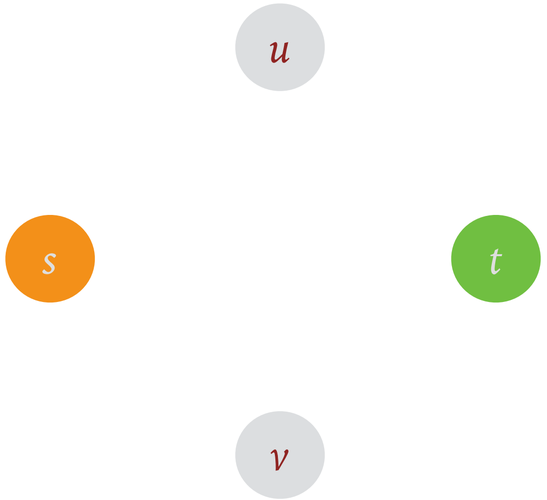


What is the resulting flow
if the current flow is augmented
over $P = s - v - u - t$?









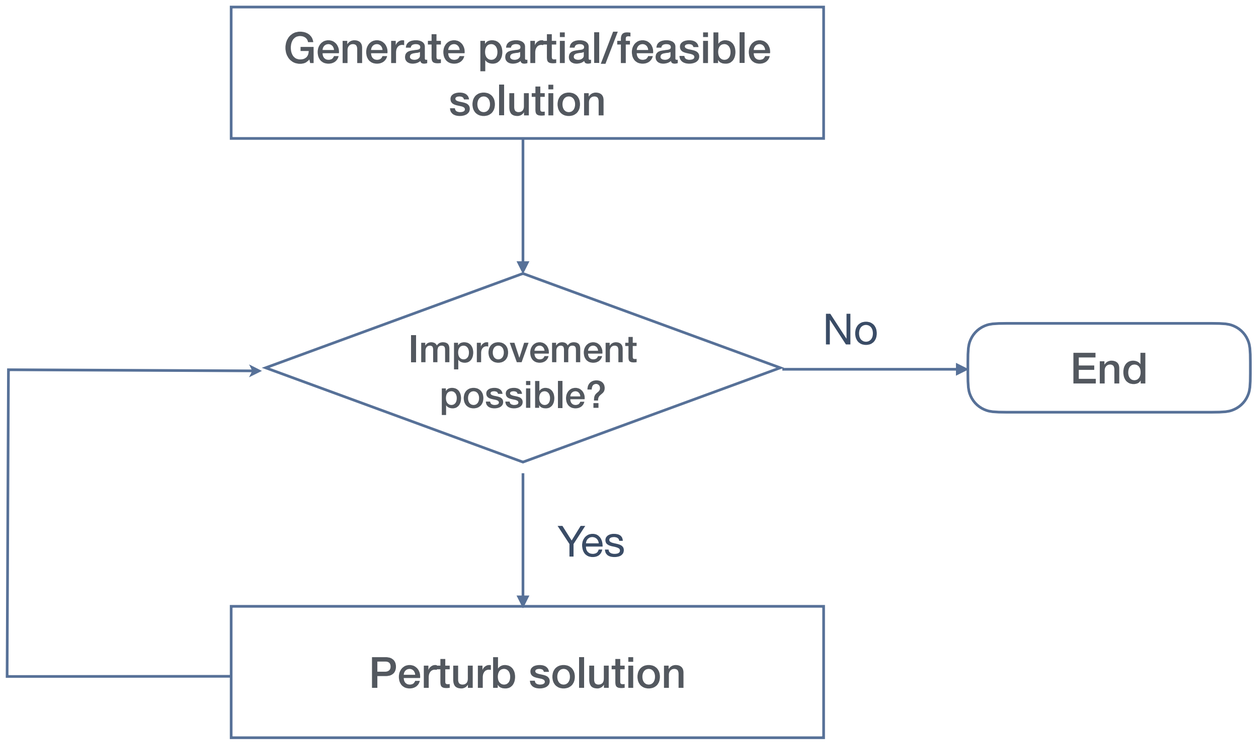
Augmenting Path or (Basic) Ford-Fulkerson Method

Method Ford-Fulkerson

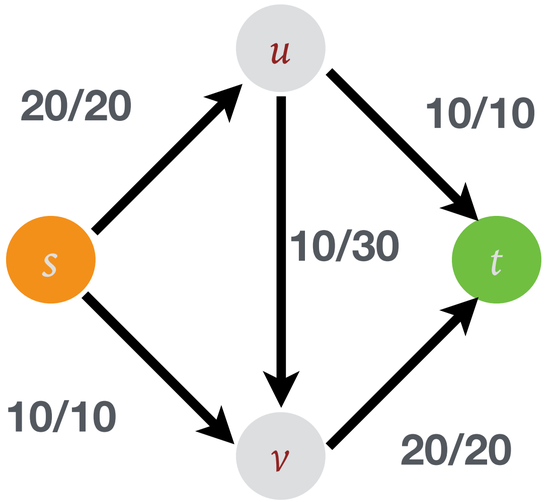
1: Input: Flow network $\boldsymbol{G} = (\boldsymbol{V}, \boldsymbol{E}, s, t, c)$
2: Output: Maximum flow f
3:
4: Initialize $f(e) = 0$ for all edges
5: **while** there exists a simple $s \rightsquigarrow t$ path P in the residual graph $\boldsymbol{G}_f$ **do**
6: $\quad f \leftarrow \text{Augment}(f, P)$
7: **return** f

How do we know that the method yields the maximum flow ?
More on this later.

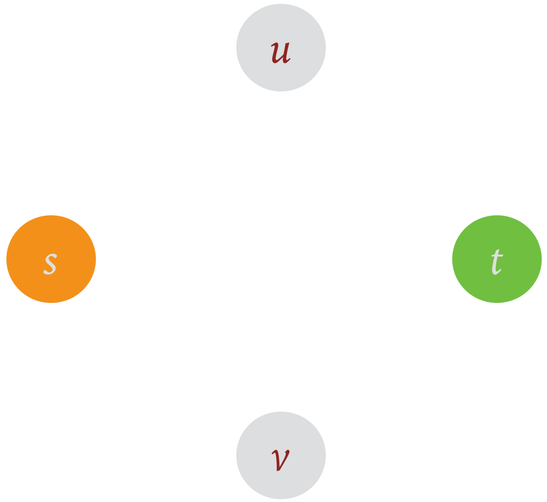
Iterative Improvement Design Strategy



Graph G with flow f



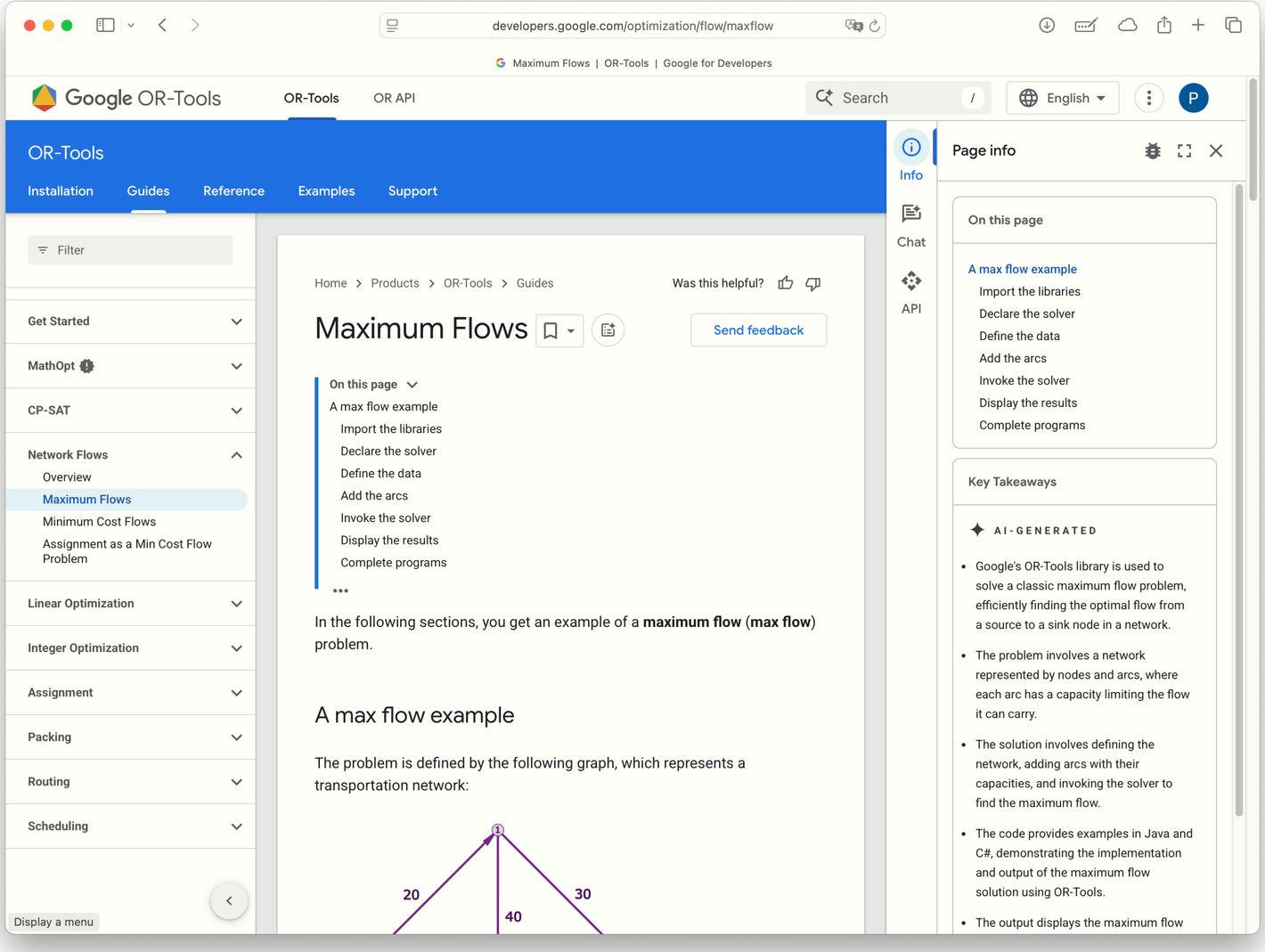
Residual graph G_f



In-Class Assignment L5-1 (Group of Three or Less): Ford-Fulkerson Method

(25 minutes) Submit to LEB2

https://developers.google.com/optimization/flow/maxflow



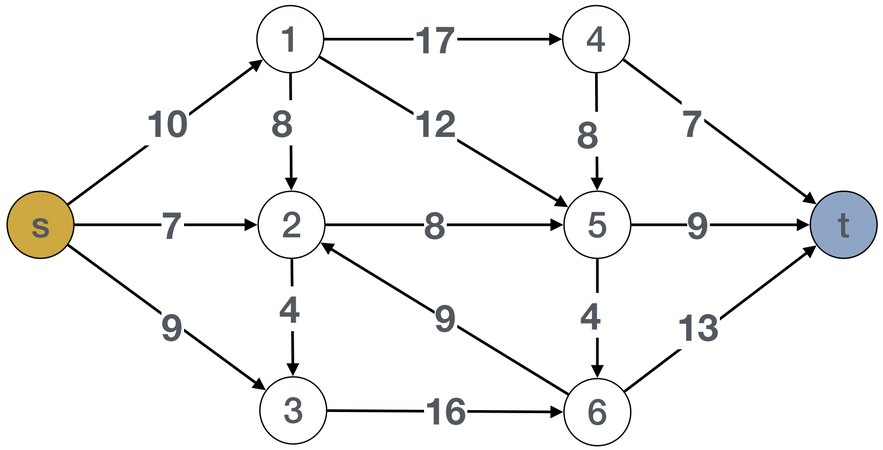
Cut

An ***st-cut*** or ***cut*** $C(X, \overline{X})$ is a ***set of all edges*** induced by vertex partitioning of the nodes such that

- $s \in X$ and $t \in \overline{X}$, where $\overline{X} = V - X$
- All the edges have tail in X and head in $\overline{X}$ (Source side to Sink side)

Example: what are $\overline{X}$ and $c(X, \overline{X})$ for the following X ?

- $X = \{s,1\}$
- $X = \{s,2,3\}$
- $X = \{s,1,2,3\}$



Cut Capacity

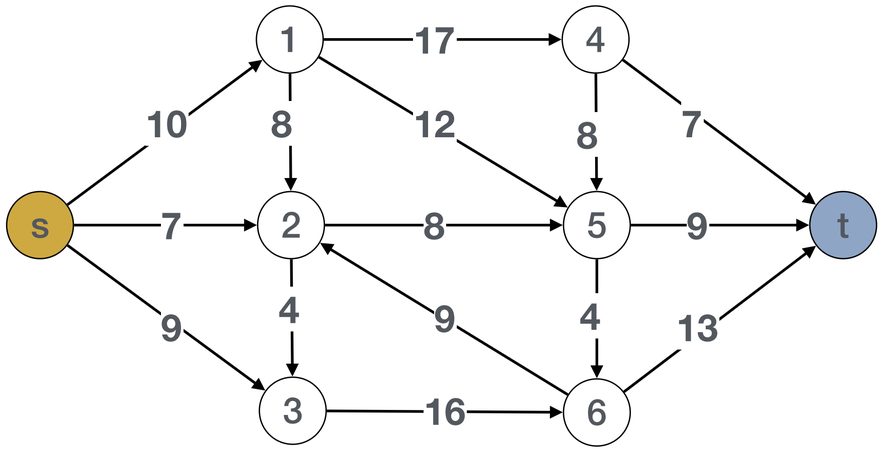
The ***cut capacity*** $c(\boldsymbol{X}, \overline{\boldsymbol{X}})$ is the sum of the ***edge capacities*** (u, v), $u \in \boldsymbol{X}$, $v \in \overline{\boldsymbol{X}}$

Minimum cut is the cut with minimum capacity.

Net flow of/across the cut = Flow out of $\boldsymbol{X}$ - Flow into $\boldsymbol{X}$.

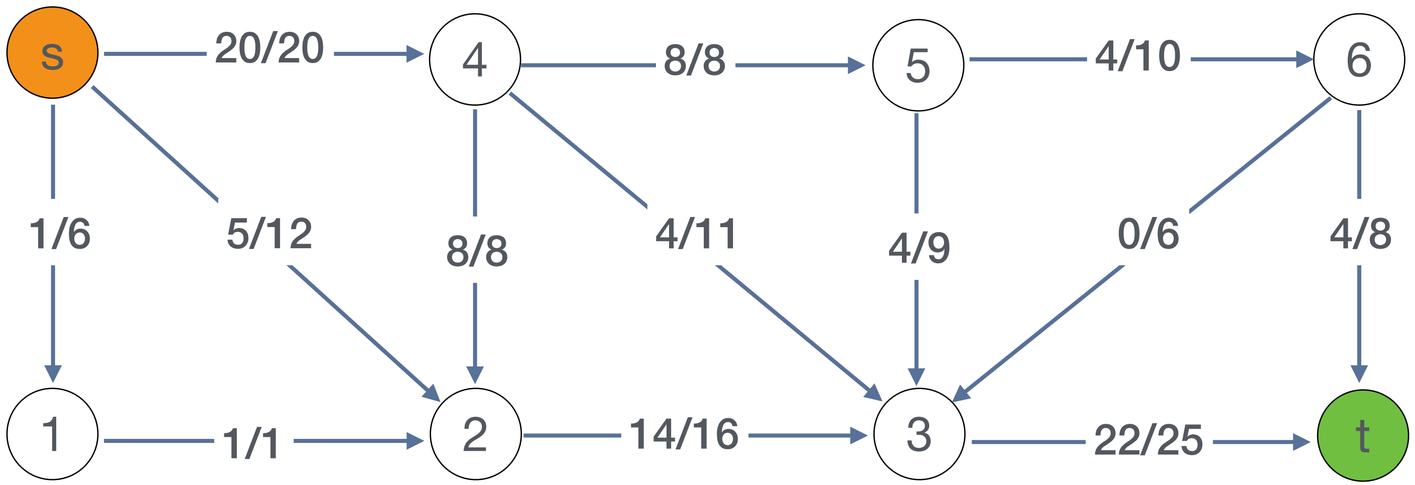
Example: If $\boldsymbol{X} = \{s,2,3\}$,

- What is the cut capacity of $c(X, \overline{X})$?
- What is the net flow across the cut?



Self Test: Cut and Cut Capacity

https://forms.office.com/r/nvkAGS9dvz?origin=lprLink

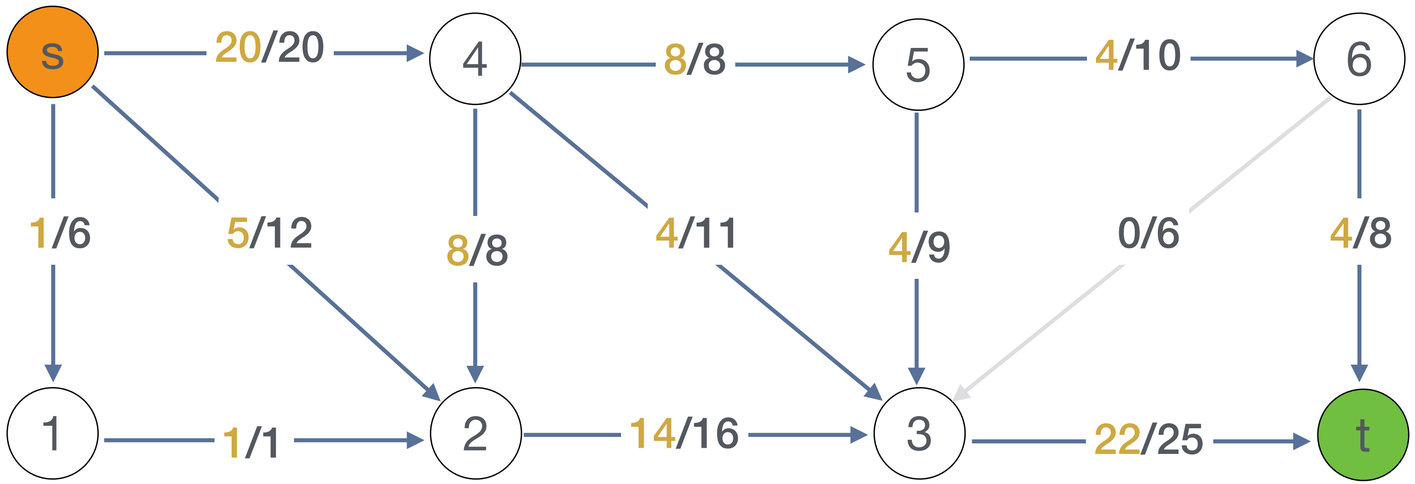

s
20/20
4
8/8
5
4/10
6
1/6
5/12
8/8
4/11
4/9
0/6
4/8
1
1/1
2
14/16
3
22/25
t

Relationship of Cut and Flow

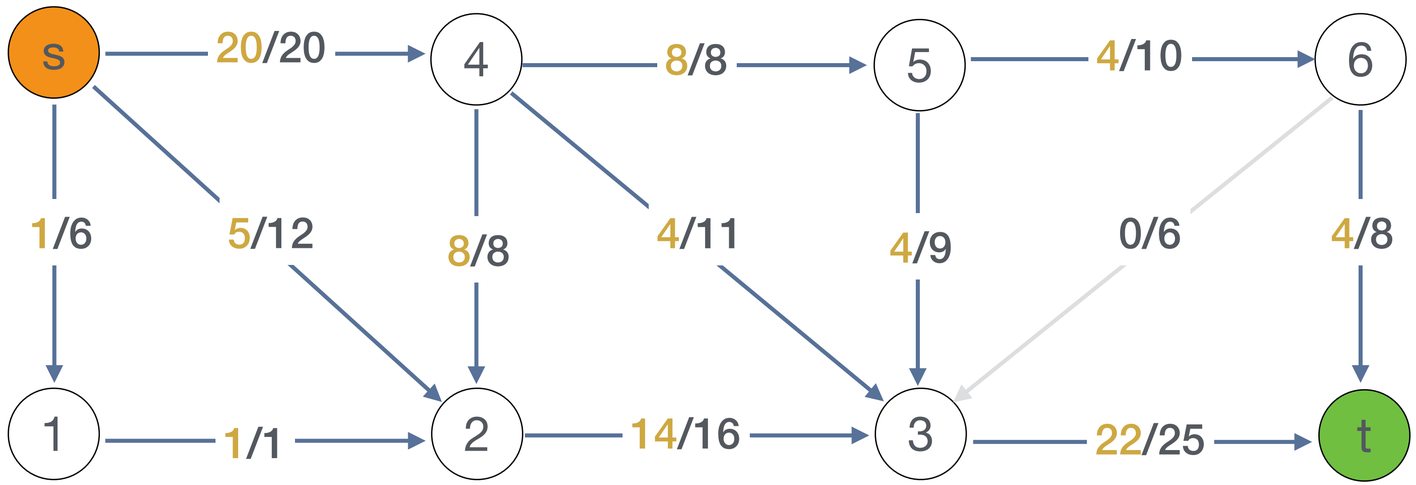
Fact 1: Flow value lemma

- Let f be ***any st-flow*** and let $C(X, \overline{X})$ be ***any cut***.
- Then, the value of the flow f equals the ***net flow*** of/across the cut $C(X, \overline{X})$.

$$v(f) = \sum_{e \text{ out of } X} f(e) \quad - \sum_{e \text{ into } X} f(e)$$

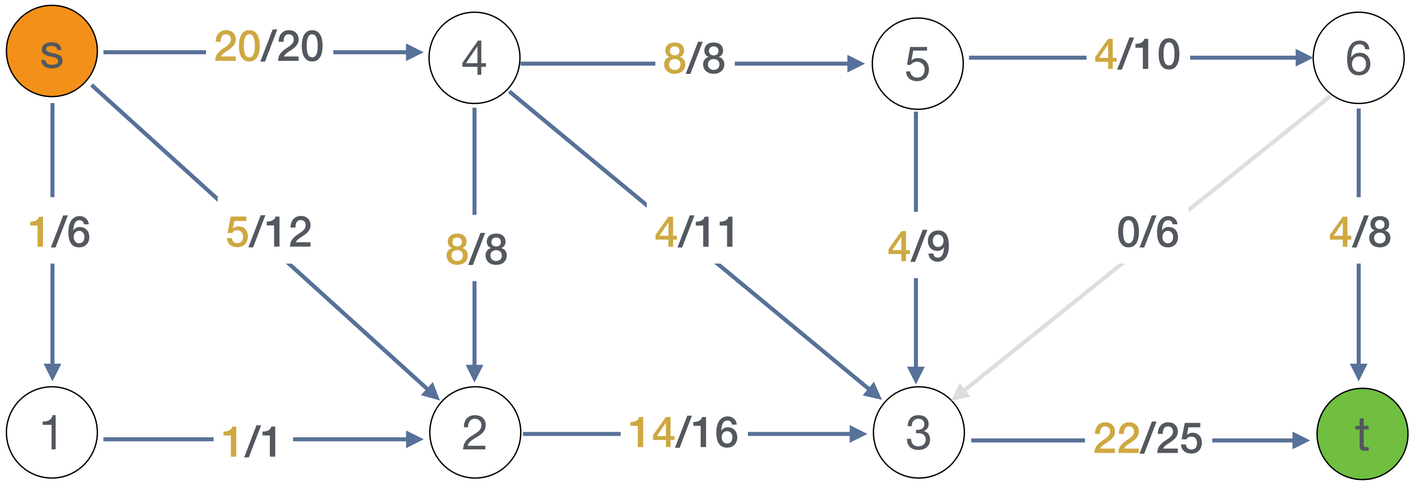


Fact 2: Let f be ***any st-flow*** and let $C(X, \overline{X})$ be ***any cut***. Then, $v(f) \leq c(X, \overline{X})$.



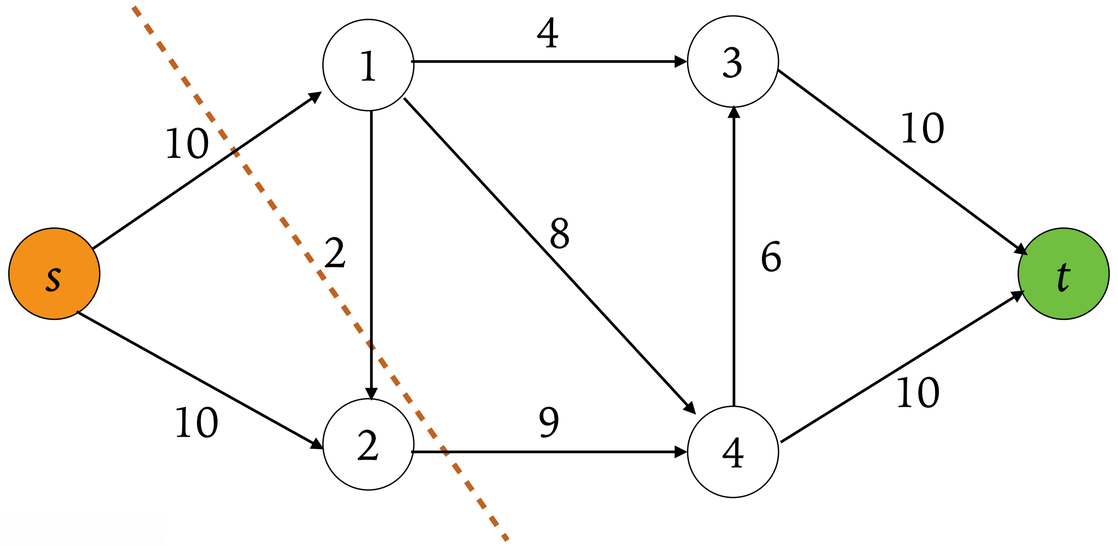
Fact 3: Suppose there is an ***st-flow*** f^* and cut $C(X^*, \overline{X}^*)$ such that $v(f^*) = c(X^*, \overline{X}^*)$. Then,

- f^* is a maximum flow.
- $C(X^*, \overline{X}^*)$ is a minimum cut, or min-cut.



Max-Flow Min-Cut Theorem

The value of a maximum flow in a network is equal to the capacity of its minimum cut.



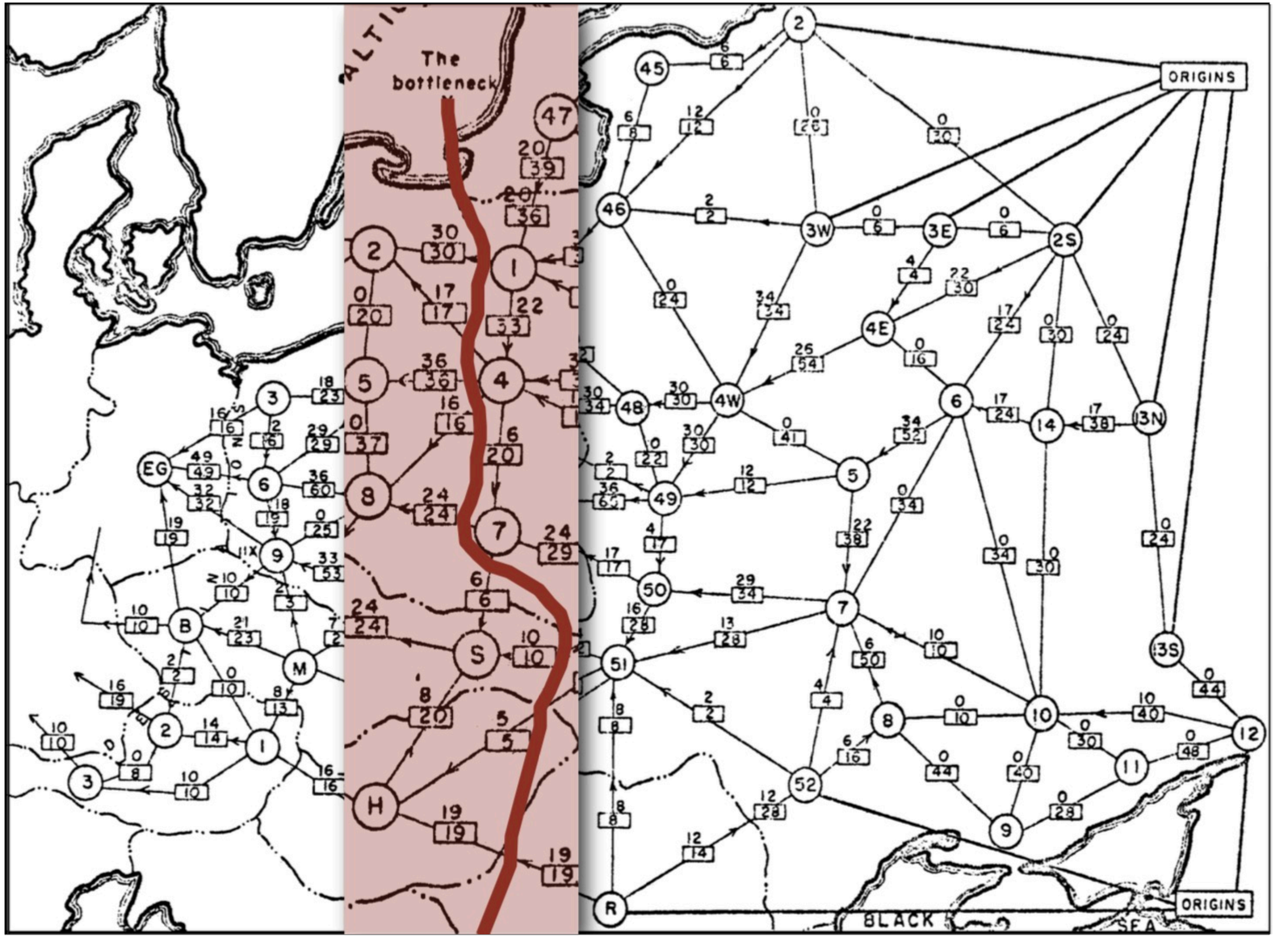
MAXIMAL FLOW THROUGH A NETWORK

L. R. FORD, JR. AND D. R. FULKERSON

Introduction. The problem discussed in this paper was formulated by T. Harris as follows:

"Consider a rail network connecting two cities by way of a number of intermediate cities, where each link of the network has a number assigned to it representing its capacity. Assuming a steady state condition, find a maximal flow from one given city to the other."

"Free world" goal. Cut supplies (if cold war turns into real war).

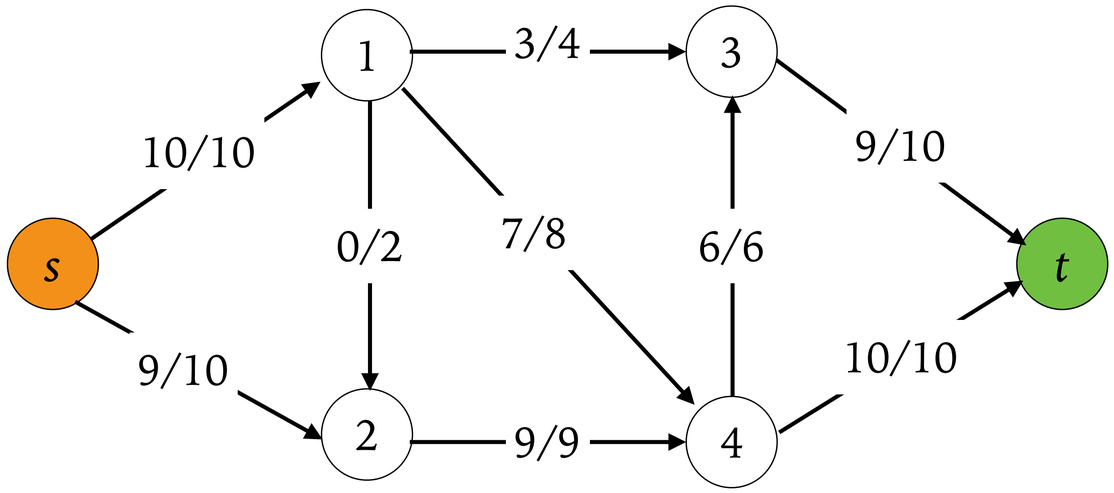


Harris and Ross's map of the Warsaw Pact rail network

Theorem: The *st*-flow returned by Ford-Fulkerson Method is a Maximum Flow.

Basically, the residual graph yields in an *st*-flow value equal to the cut capacity. So, it is the max flow.

See proof in the supplementary slides.



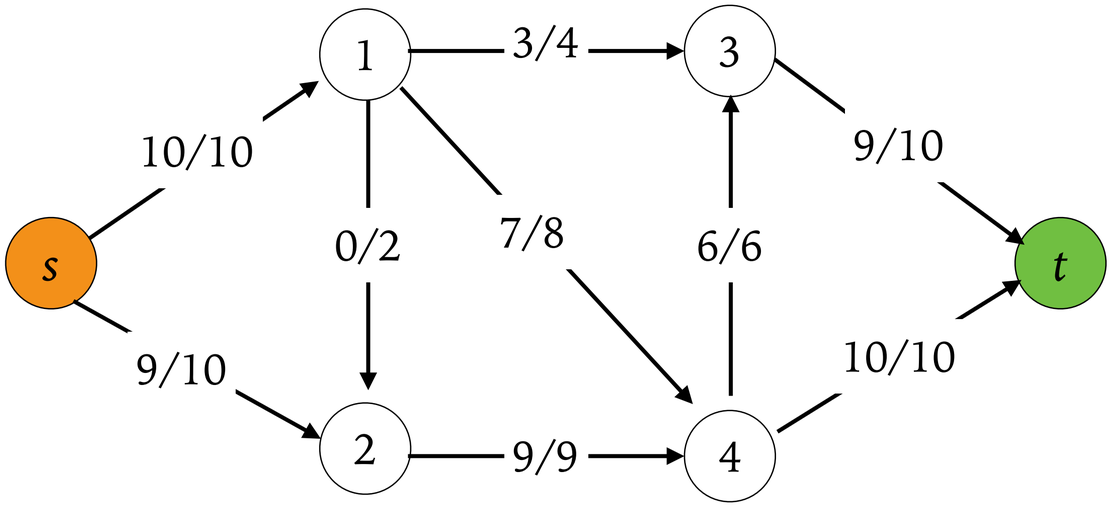
Max-Flow Solution Properties

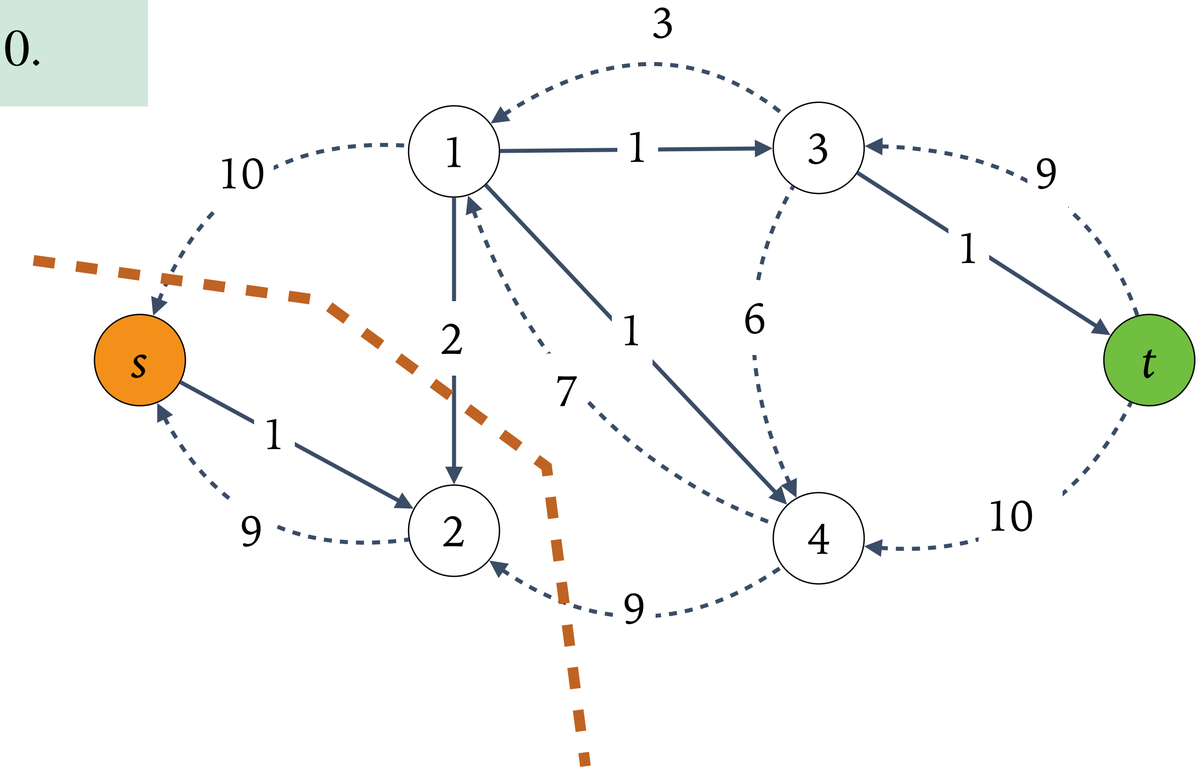
Let G_f be a residual graph obtained from running Ford-Fulkerson method.

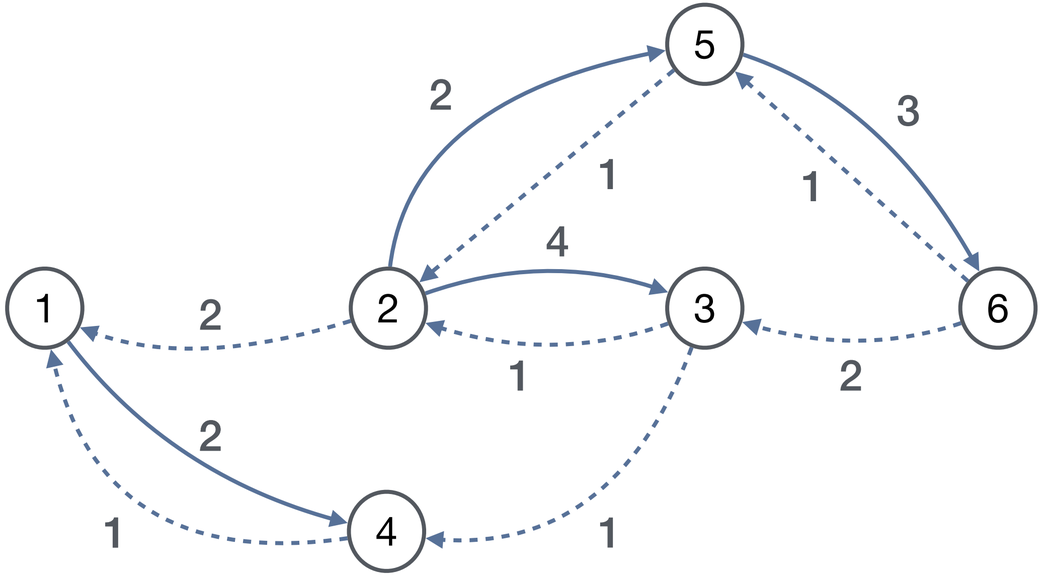
Let $C(X^*, \overline{X}^*)$ be the st-cut where X^* contains all nodes reachable from s in G_f

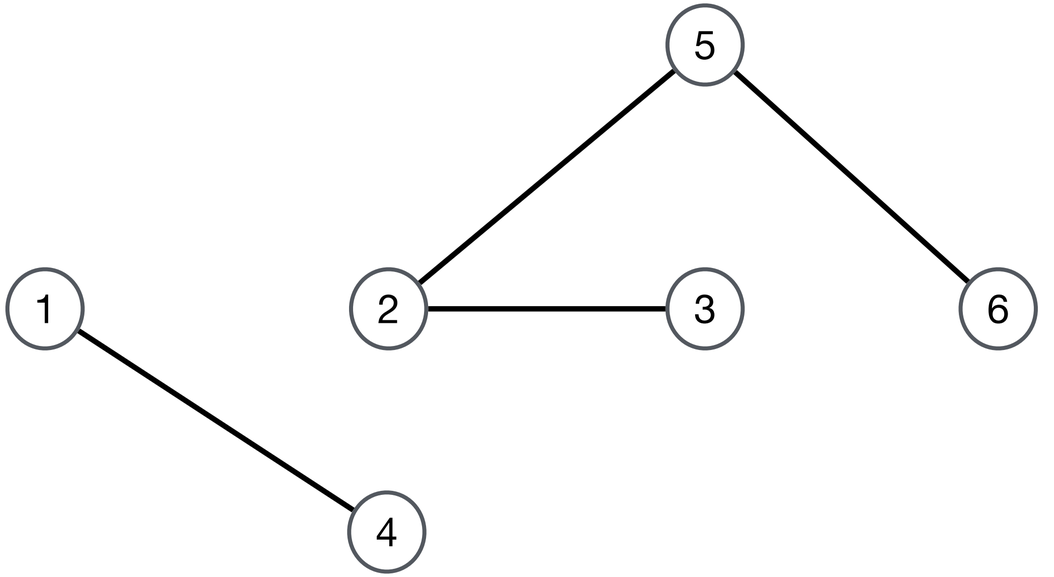
I. Any edge out of X^* has $f(e) = c(e)$.

II. Any edge into X^* has $f(e) = 0$.

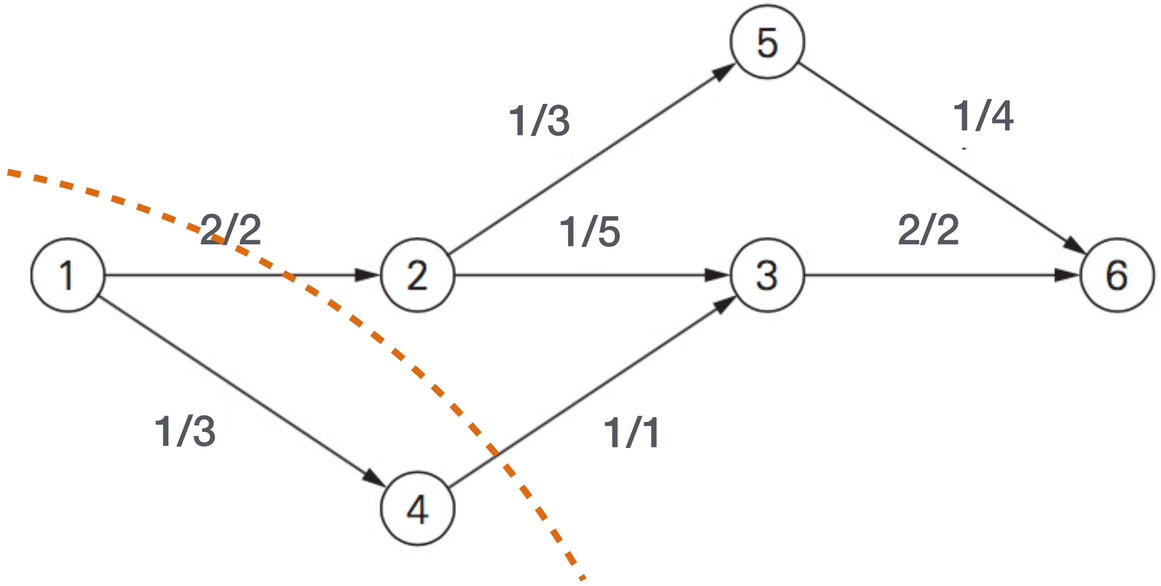








How to obtain a min-cut from a max-flow ?



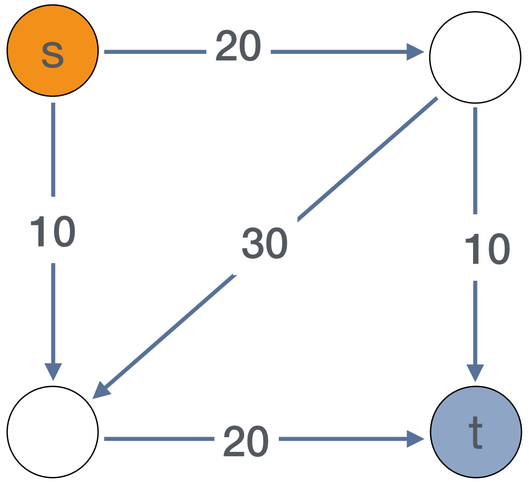
Min-cut capacity = 3 = Maximum flow value

Running Time of Ford-Fulkerson Algorithm

Algorithm Ford-Fulkerson

Input: Flow network $\boldsymbol{G} = (\boldsymbol{V}, \boldsymbol{E}, s, t, c)$
Output: Maximum flow f

Initialize $f(e) = 0$ for all edges
while there exists a simple $s \rightsquigarrow t$ path P in the residual graph $\boldsymbol{G}_f$ **do**
    $f \leftarrow \text{Augment}(f, P)$
return f



$$O(C(|V| + |E|)), \text{ where } C = \sum_{e \text{ out of } s} c(e)$$

Integrality Theorem: If every edge capacity is an integer $(1 \leq c(e) \leq C)$, there exists an integral max flow (all $f(e)$ are integers).

Choosing Good Augmenting Paths

FF performance depends on choice of augmenting paths.

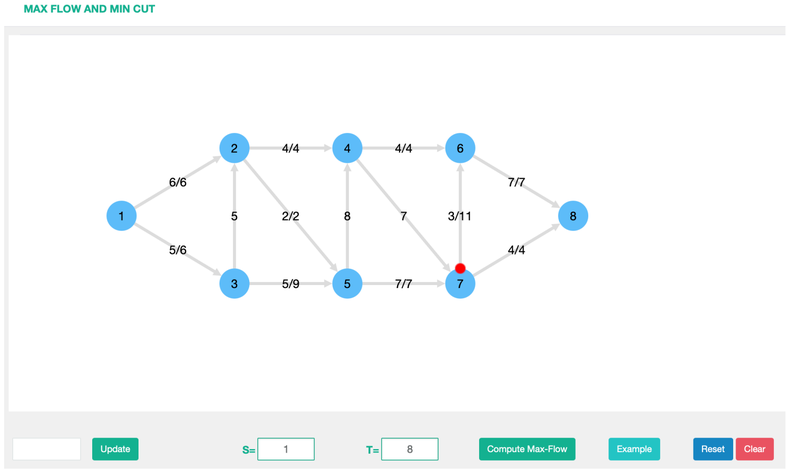
Method	Number of paths	Implementation
Shortest path	≤ 1/2 |E| |V|	Breadth First Search
Fattest path	≤ |E| ln (|E| U)	Priority queue

Edmonds-Karp algorithm $O(|V||E|^2)$

$1 \leq f(e) \leq U$

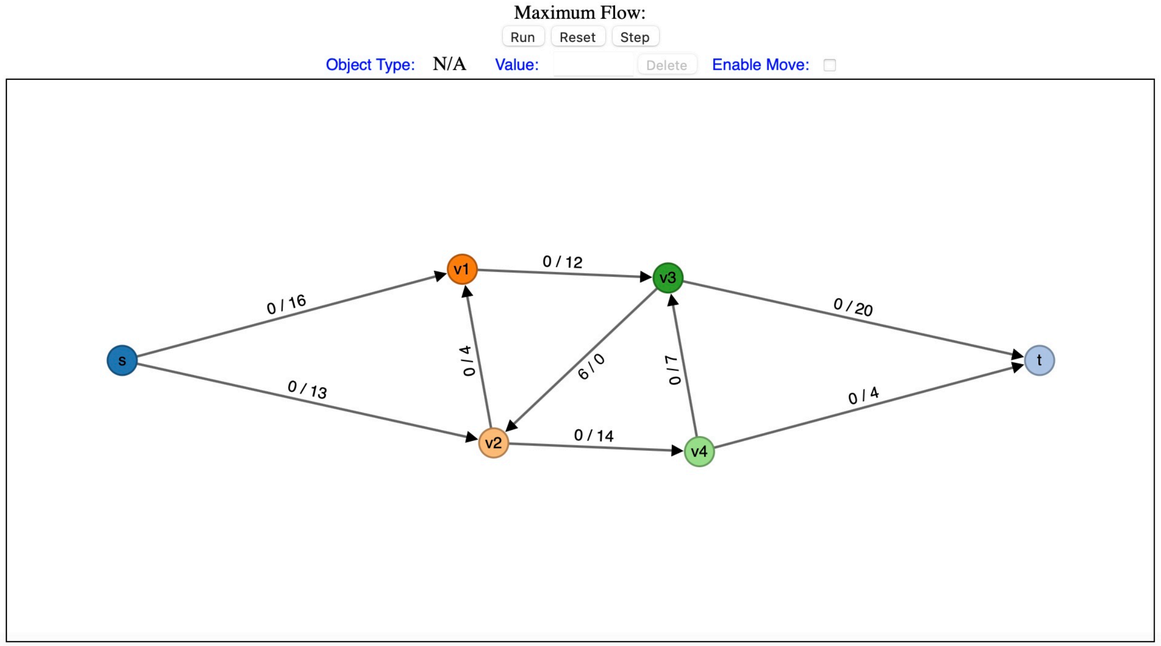


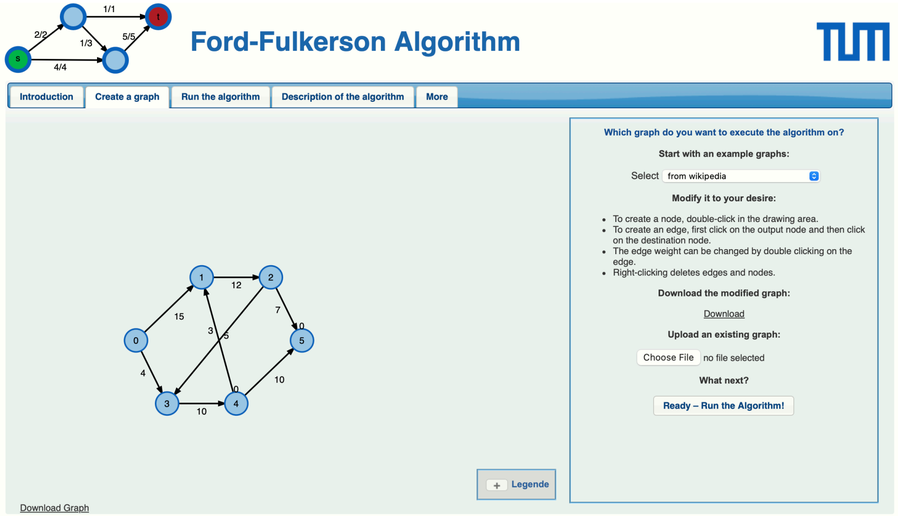
https://isabek.github.io/



Edmonds-Karp Algorithm

Visualization



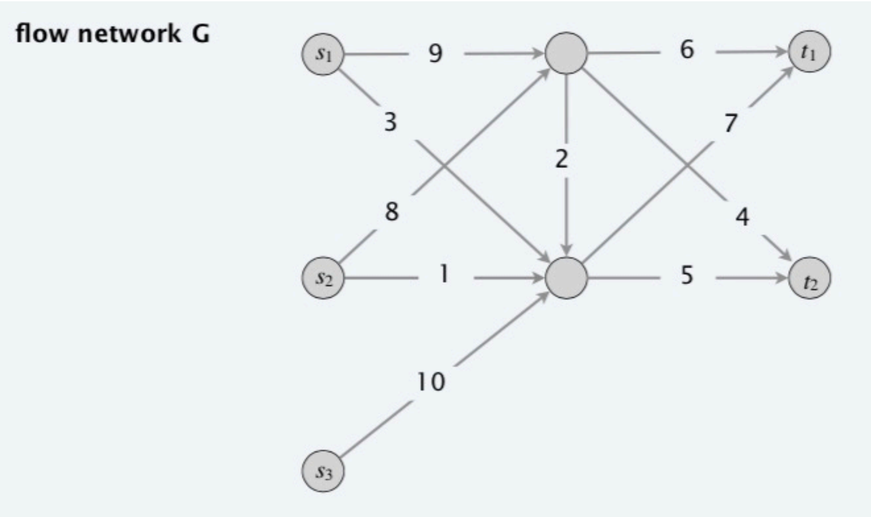


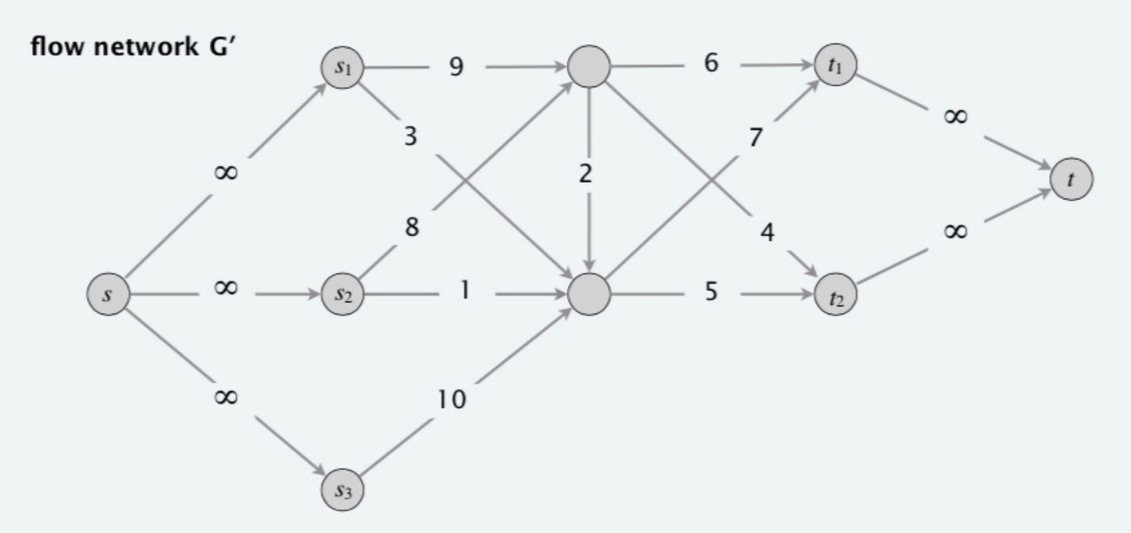
Step by step visualization of flow augmentation using residual graphs.

https://jamieheller.github.io/editor.html

https://algorithms.discrete.ma.tum.de/graph-algorithms/flow-ford-fulkerson/index_en.html

Variants of Max-Flow Problem: Multiple Sources and Sinks





Some Network Flow Applications

Carpool sharing

Exam scheduling

Circulation problem

Carpool Sharing Problem

On each day, only some participants join the carpool, and one of those participants must be selected as the driver. How to distribute driving responsibility fairly according to how often each person participates ?

On a day where k people use the carpool, each will receive $1/k$ share of responsibility to drive.

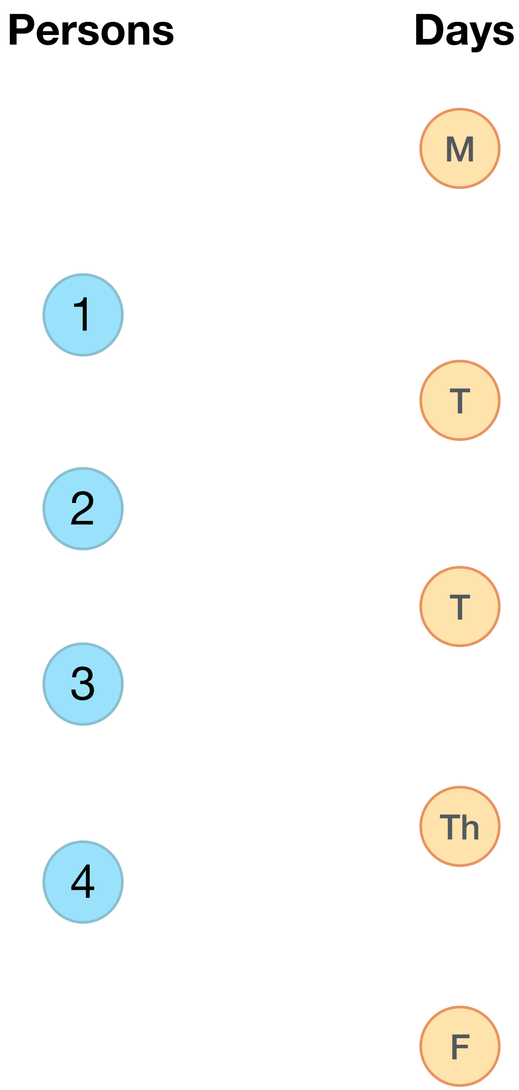
The total share r_i of the person i is the sum of daily shares.

Person i drives at most $\lceil r_i \rceil$ times in that week.

	M	T	W	Th	F
Person 1	✓	✓	✓		
Person 2	✓		✓		
Person 3	✓	✓	✓	✓	✓
Person 4		✓	✓	✓	✓

$r_1 =$
$r_2 =$
$r_3 =$
$r_4 =$

Ref: David P. Williamson, Network Flow Algorithms, 2019 (Section 2.2)

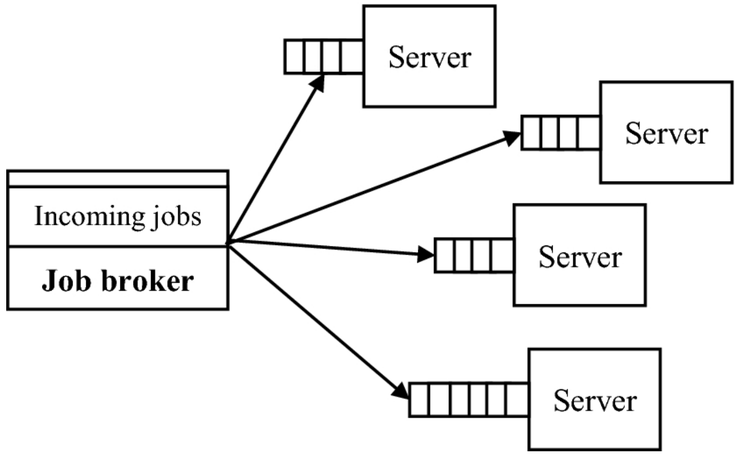

Persons
Days
M
1
T
2
T
3
Th
4
F

Job Computing Assignment

N jobs arrive to a scheduler with M servers.

- Based on the job requirement (GPU, RAM size), only certain servers are capable of running a given job.
- Jobs can be split among the servers.

The scheduler needs to distribute jobs appropriately to the servers.



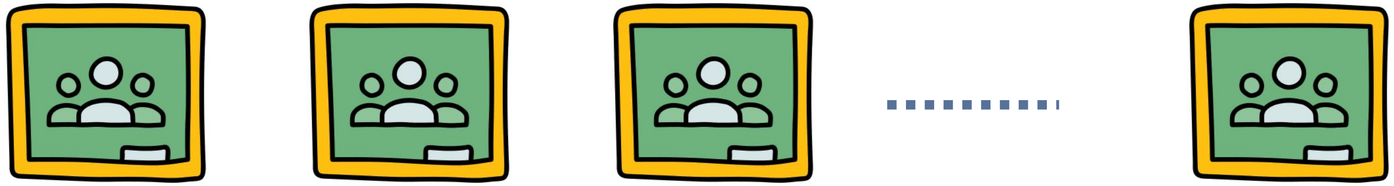
	S1	S2	S3	S4	S5	S6	S7
Job 1	✓	✓	✓				✓
Job 2	✓		✓			✓	✓
Job 3	✓	✓	✓	✓	✓	✓	
Job 4		✓	✓	✓	✓		

$$r_1 = 17/12 \approx 2$$
$$r_2 = 19/12 \approx 2$$
$$r_3 = 29/12 \approx 3$$
$$r_4 = 19/12 \approx 2$$

Exam Scheduling Problem



N classes to schedule for final exams.
Class i has $E[i]$ students enrolled.



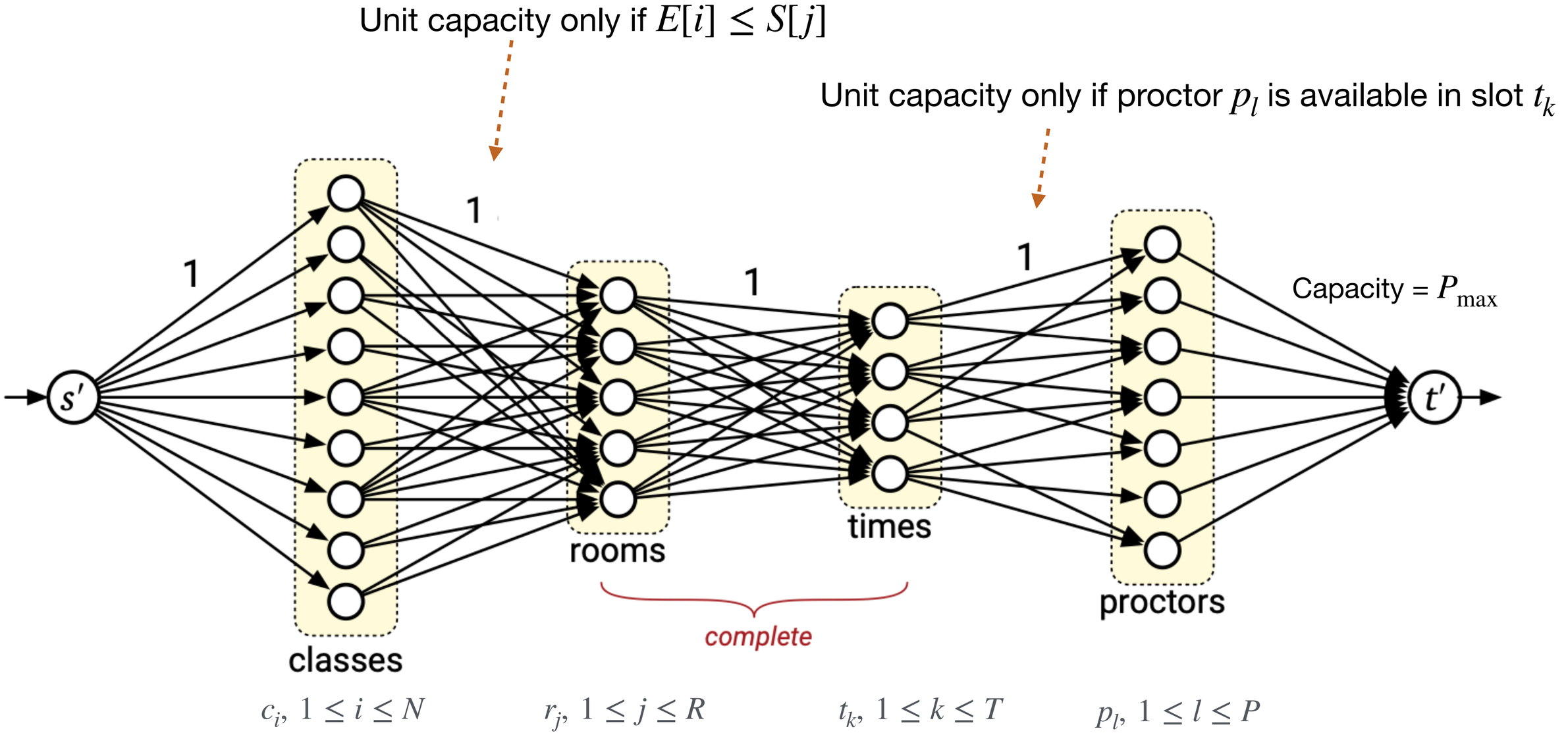
R rooms available.
Room j has $S[j]$ seats.



T time slots



P proctors available in only selected time slots.
Each proctor cannot oversee more than $P_{\max}$ exams.



Ref: J. Erickson, Algorithms, 2019 (Ch.11 Applications of Flows and Cuts)
https://jeffe.cs.illinois.edu/teaching/algorithms/

Circulation Problem

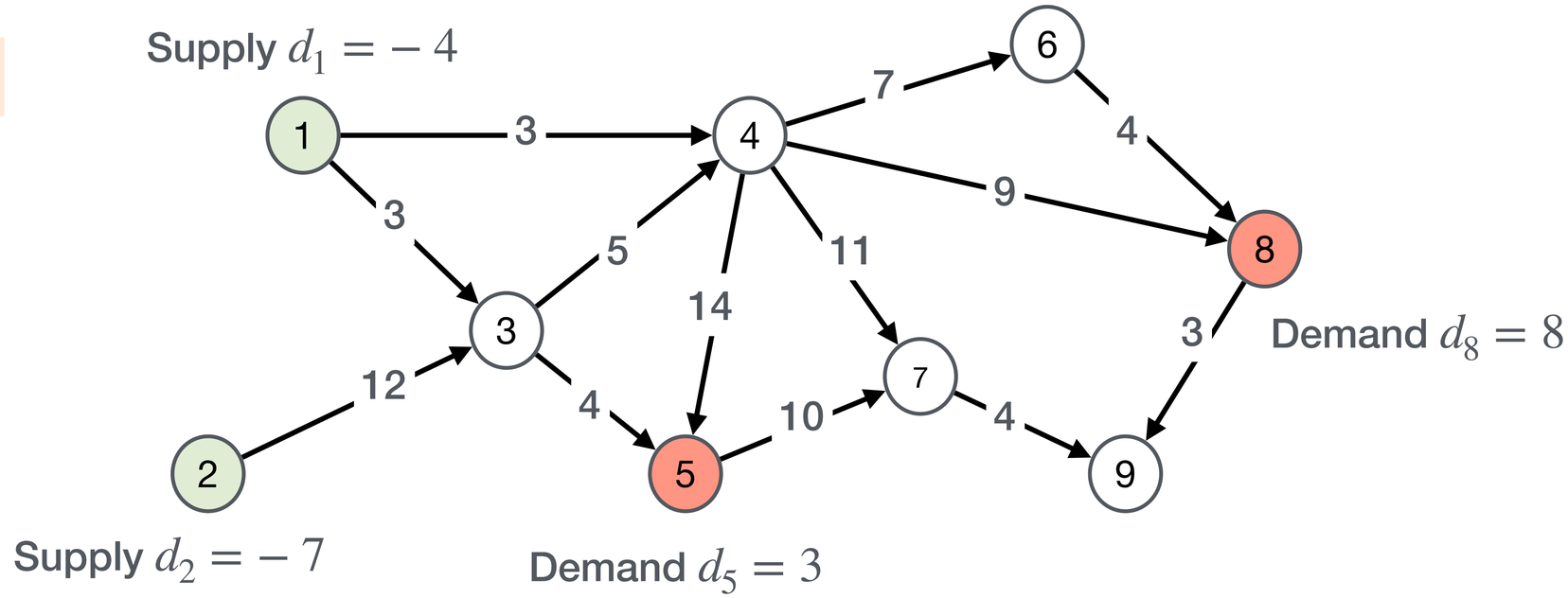
Define a circulation network $\boldsymbol{G} = \langle \boldsymbol{V}, \boldsymbol{E}, d_v, c \rangle$ with

- A set of supply nodes $\boldsymbol{S}$ with integer supply $d_v < 0$, $\forall v \in \boldsymbol{S} \subset \boldsymbol{V}$
- A set of demand nodes $\boldsymbol{T}$ with integer demand $d_v > 0$, $\forall v \in \boldsymbol{T} \subset \boldsymbol{V}$
- Remaining nodes with $d_v = 0$ (Transhipment node)
- Edge capacity $c(e)$

Flow network for circulation problem

Three node types

- Supply node
- Demand node
- Transhipment node



Feasible Circulation

A ***circulation*** with demands $\{d_v\}$ is a ***function*** $f(e)$ that assigns a nonnegative real number to each edge in $\boldsymbol{G}$ and satisfies the following two conditions:

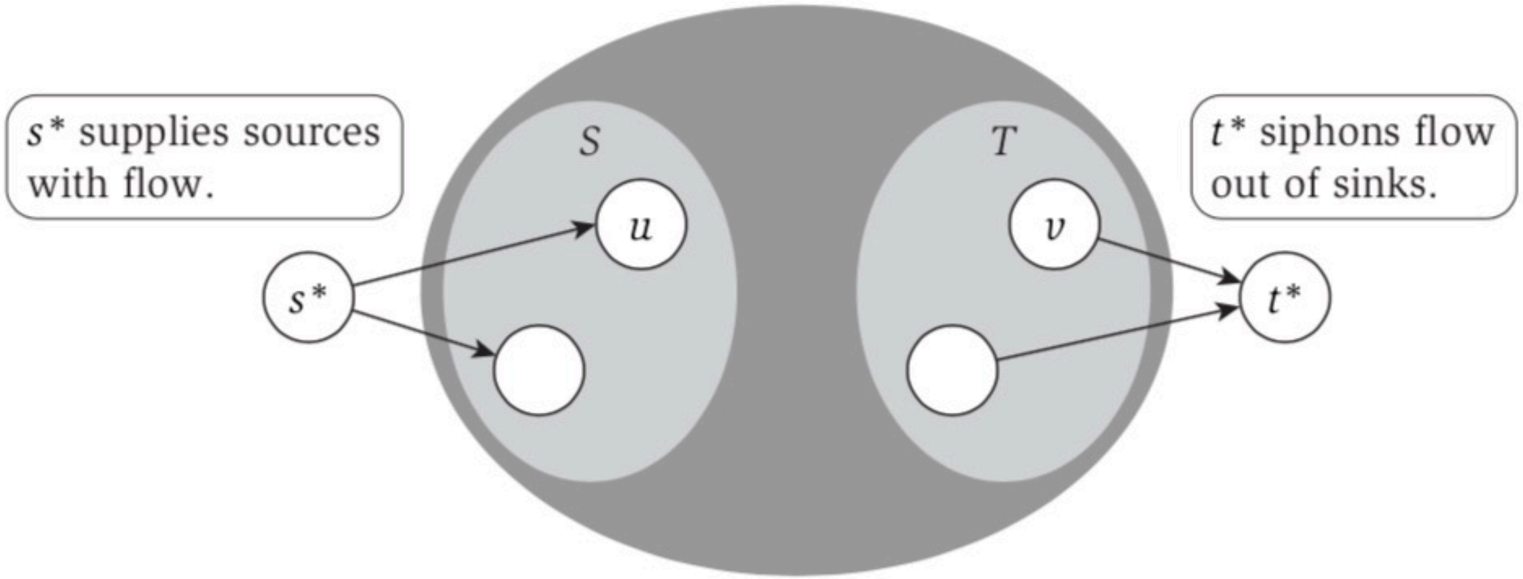
- **Capacity conditions**: Assignment of $f(e)$ such that $0 \leq f(e) \leq c(e)$ for $\forall e \in \boldsymbol{E}$
- **Demand conditions:** Requirement at node $v \in \boldsymbol{V}$:

$$\sum_{e \text{ into } v} f(e) \; - \sum_{e \text{ out of } v} f(e) = d_v$$

Goal: Determine if there exists such feasible circulation in $\boldsymbol{G}$.

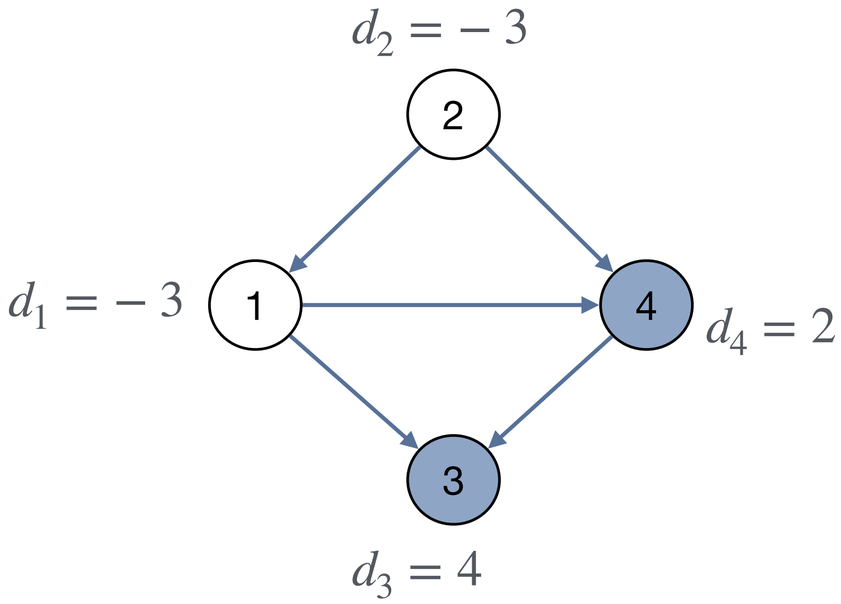
Reduction to Max-Flow Problem

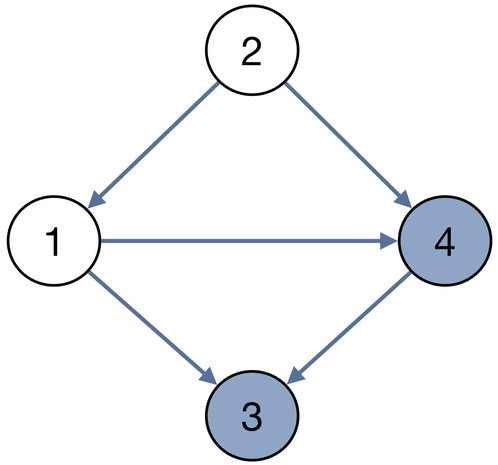
Add super-source and super-sink nodes to transform $\boldsymbol{G}$ to $\boldsymbol{G}'$.



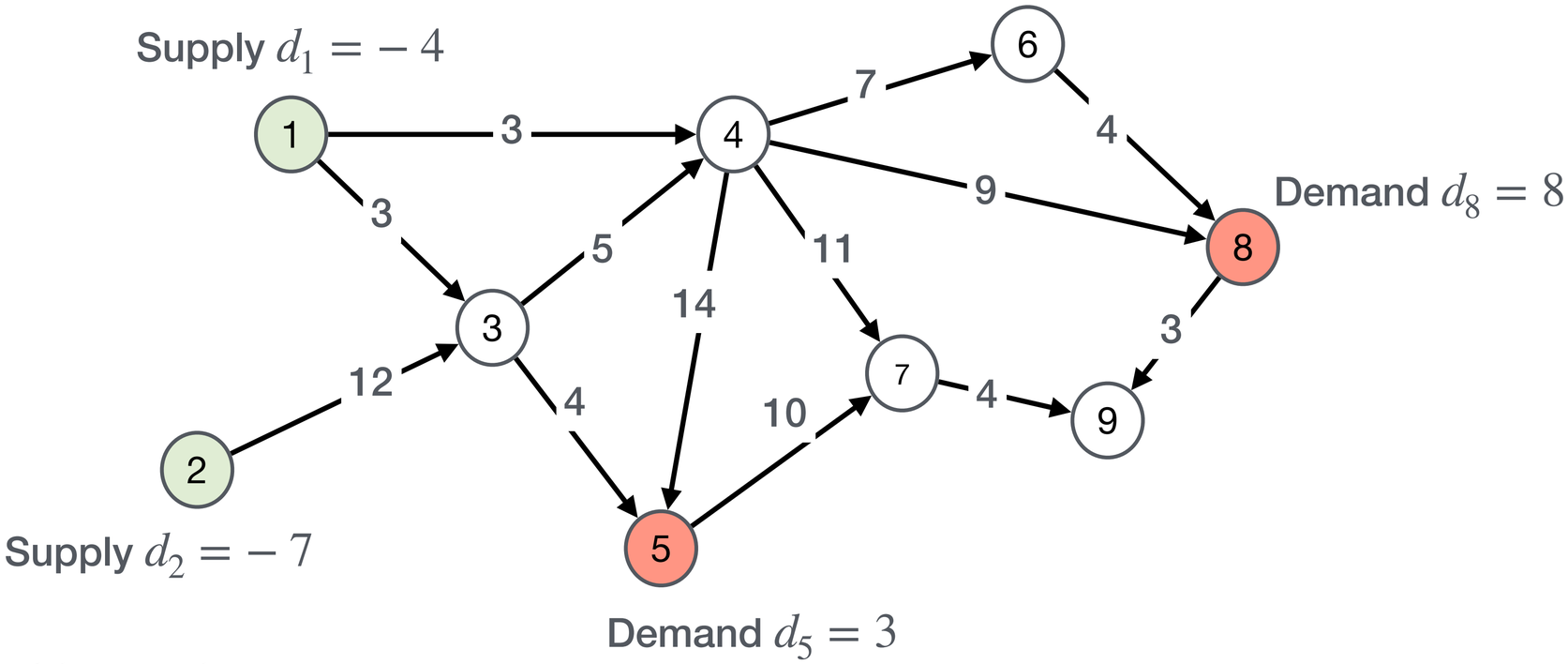
Edge capacities from s^* and to t^* are $|d_v|$.

Example





Circulation Problem as Max-Flow Problem



When does G has a feasible circulation ?

What if Σ supplies $>$ Σ demands ?

What if Σ supplies $<$ Σ demands ?

In-Class Assignment L5-2 (Group of Three or Less): Bike Sharing

(25 minutes) Submit to LEB2

From the given bike data and the bike-carrying capacity graph,

(a) Determine the supply and demand values at each location based on the number of available bikes and reservations.

(b) Model the problem as the max-flow problem by drawing the corresponding flow network.

(c) Solve the max-flow problem to determine the number of bikes to be transported among locations.

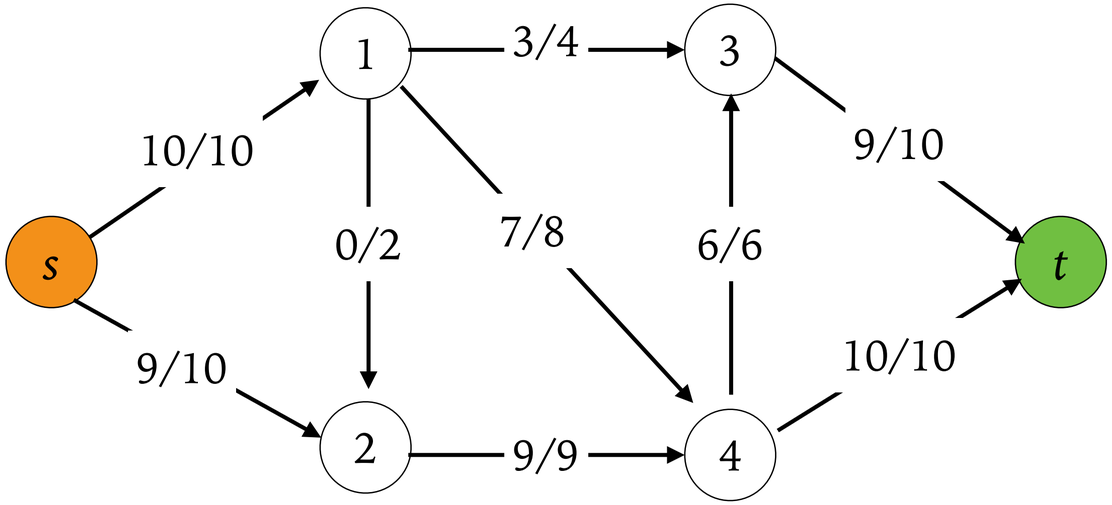
(d) Are the customer reservations satisfied at every location ? Explain.

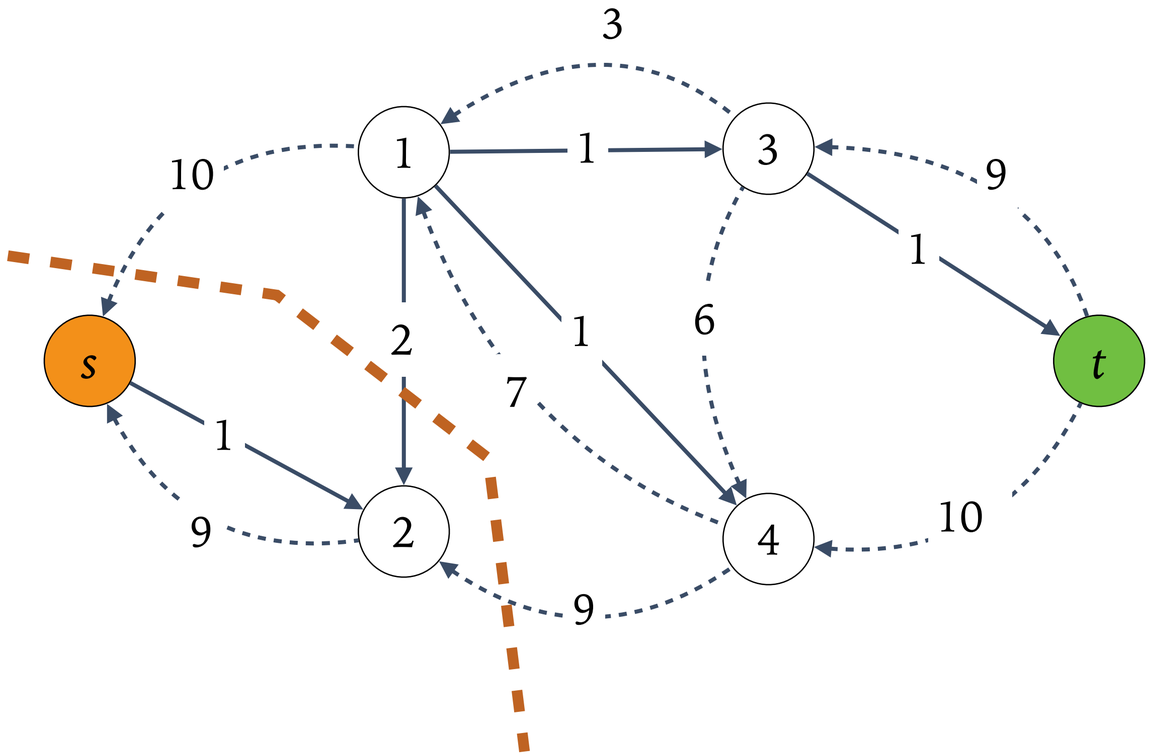
Location	Reserved	Available	Dv	Unsatisfied
A	9	17		
B	12	20		
C	12	18		
D	20	10		
E	26	15		
F	0	0		

Proof: The st-flow returned by Ford-Fulkerson Method is a Maximum Flow.

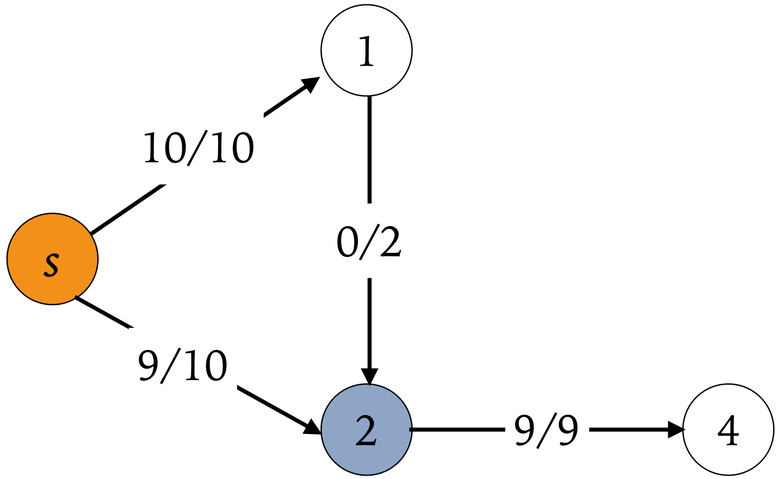
Let $\boldsymbol{G}_f$ be a residual graph obtained from running Ford-Fulkerson method.

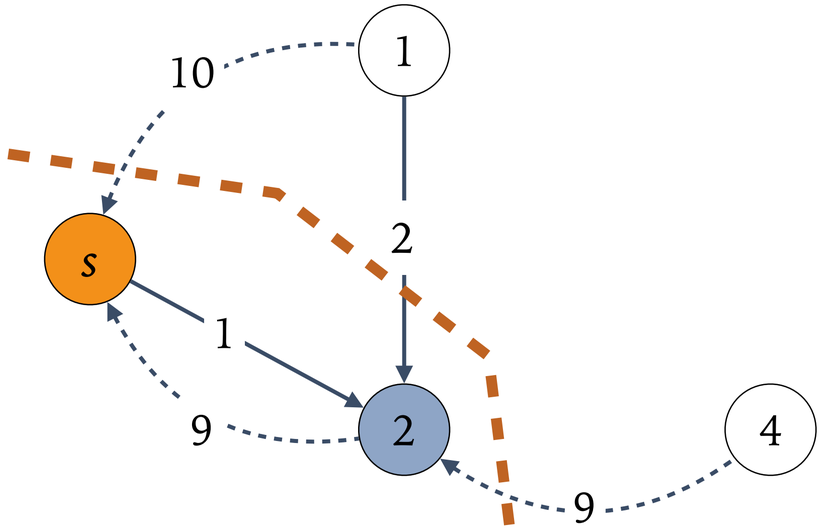
Let $C(\boldsymbol{X}^*, \overline{\boldsymbol{X}}^*)$ be the st-cut where $\boldsymbol{X}^*$ contains all nodes reachable from s in $\boldsymbol{G}_f$

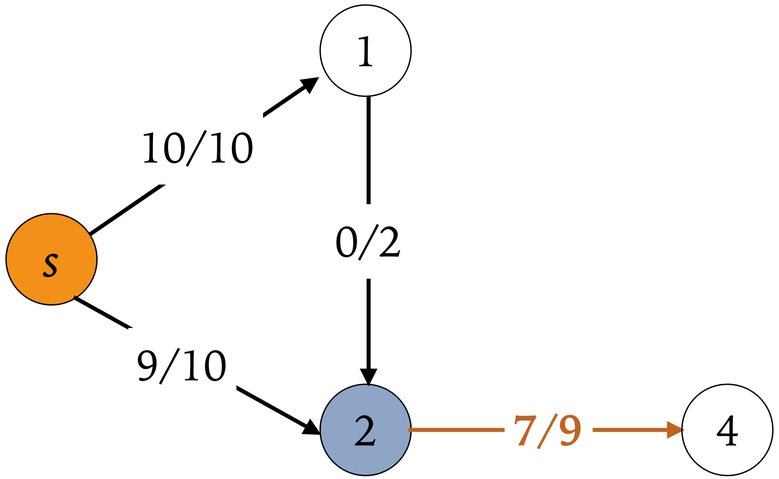




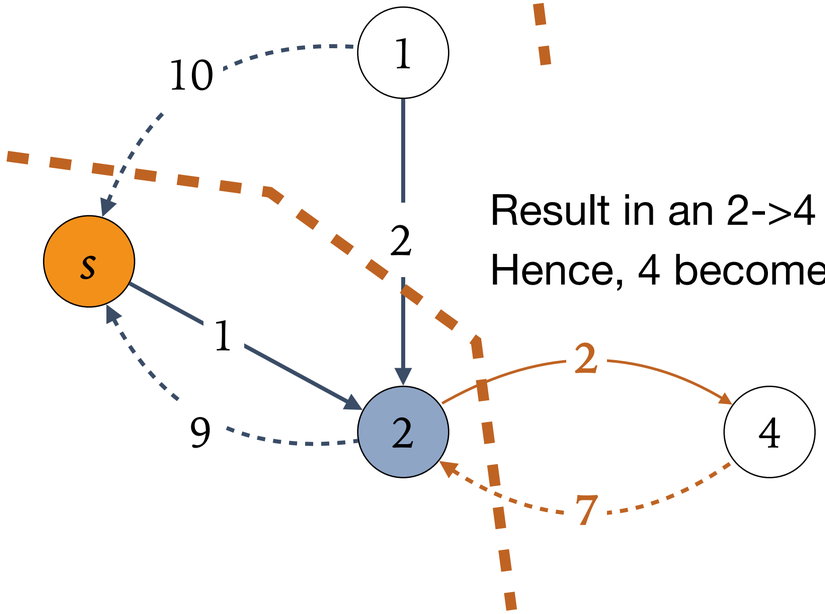
Any edge out of X^* must have $f(e) = c(e)$. What happens if some node $v \in X^*$ has an outgoing edge e with $f(e) < c(e)$?





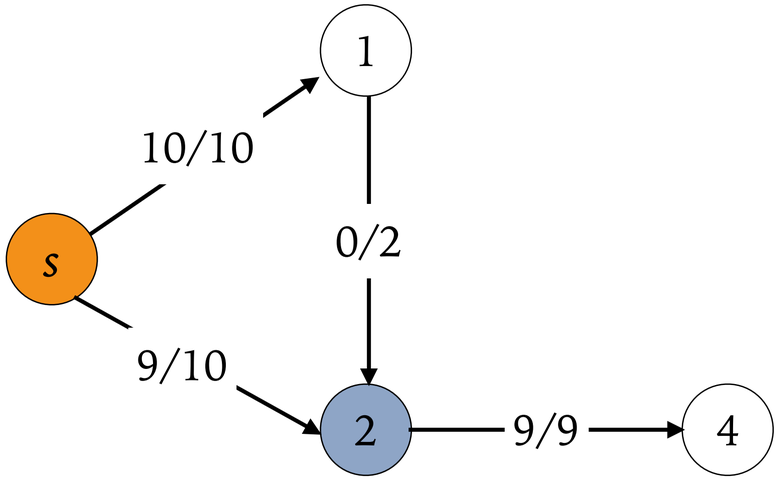


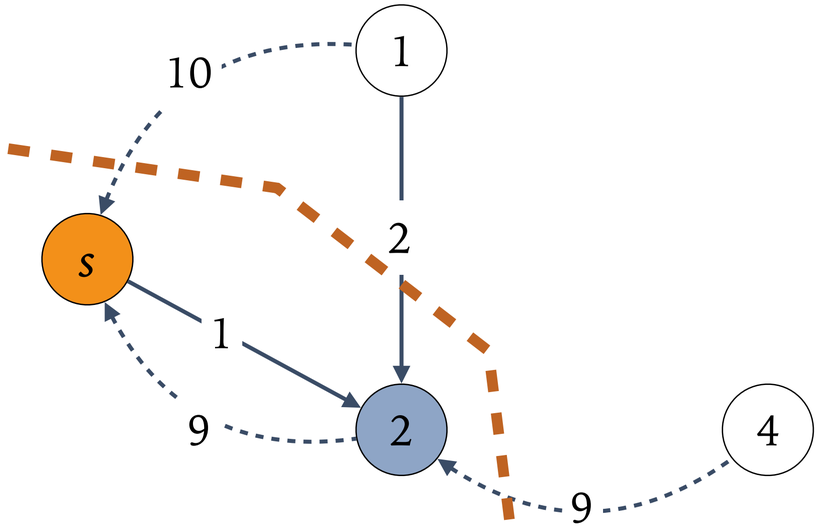
$f(e) < c(e)$

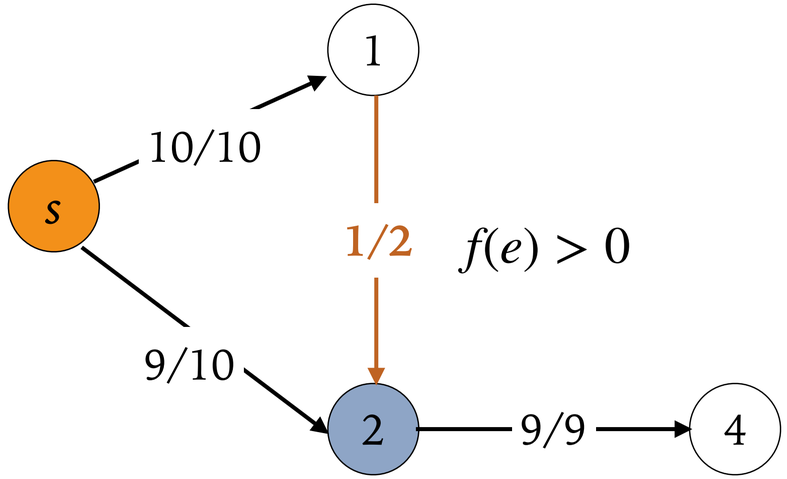


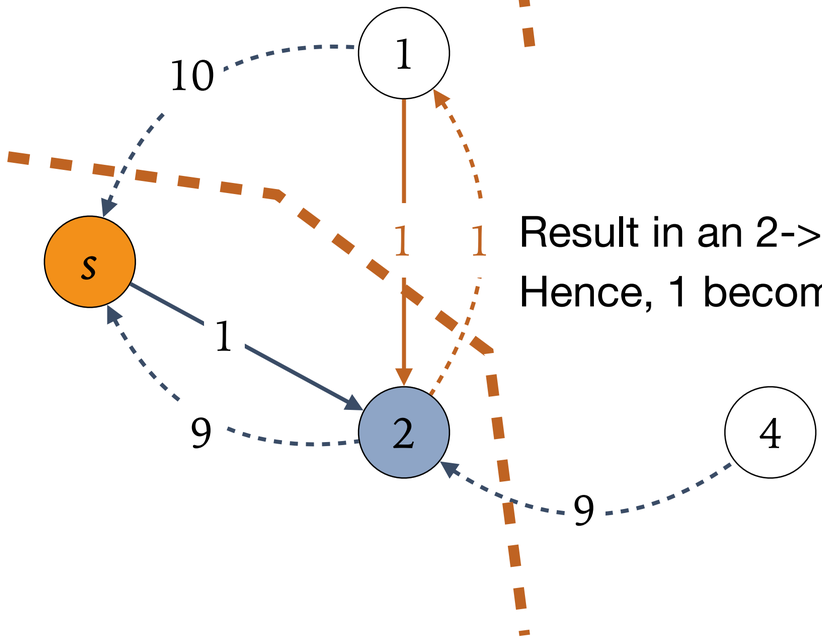
Result in an 2->4 forward edge.
Hence, 4 becomes reachable from s.

Any edge into X^* must have $f(e) = 0$. What happens if some node $v \in X^*$ has an incoming edge e with $f(e) > 0$?









Result in an 2->1 reverse edge .
Hence, 1 becomes reachable from s.

Because any edge out of X^* must have $f(e) = c(e)$, and any edge into X^* must have $f(e) = 0$. Therefore,

$$
\begin{aligned}
v(f^*) &= \sum_{e \text{ out of } X^*} f(e) \quad - \sum_{e \text{ into } X^*} f(e) \\
&= \sum_{e \text{ out of } X^*} f(e) \\
&= \sum_{e \text{ out of } X^*} c(e) = c(X^*, \overline{X^*})
\end{aligned}
$$

Because Flow value = Cut capacity, it is the max flow.

